

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ

TASK 2 CRYPTOGRAPHY

STUDENT DETAILS

NAME: ATHANASIOU VASILIOS EVANGELOS REGISTER
NUMBER: 19390005 **STUDENT**
SEMESTER: 8th **STUDENT**
STATUS: UNDERGRADUATE **PROGRAM OF**
STUDY: PADA **LABORATORY**
DEPARTMENT: ASF 05 WEDNESDAY 12:00 – 14:00
LABORATORY HEAD: GEORGOULAS ANGELOS DUE
DATE: 4/5/2023

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

STUDENT PHOTO:



ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

CONTENTS

ACTIVITIES	(PAGES 4 – 40)
1. Create a private key	(PAGES 4 – 7)
1.1 Actions	(PAGES 4 – 6)
1.2 Results 1.3	(PAGE 7)
Observations	(PAGE 7)
2. Message encryption	(PAGES 7 – 12)
2.1 Actions	(PAGES 7 – 10)
2.2 Results 2.3	(PAGES 11 – 12)
Observations	(PAGE 12)
3. Message decryption	(PAGES 12 – 16)
3.1 Actions	(PAGES 12 – 15)
3.2 Results 3.3	(PAGES 15 – 16)
Observations	(PAGE 16)
4. Message signature	(PAGES 16 – 22)
4.1 Actions	(PAGES 16 – 19)
4.2 Results 4.3	(PAGES 20 – 22)
Observations	(PAGE 22)
5. Signature Verification	(PAGES 23 – 29)
5.1 Actions	(PAGES 23 – 26)
5.2 Results 5.3	(PAGES 26 – 29)
Observations	(PAGE 29)
6. Manual verification of X.509 certificate	(PAGES 29 – 40)
6.1 Actions	(PAGES 29 – 34)
6.2 Results 6.3	(PAGES 34 – 40)
Observations	(PAGE 40)

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ACTIVITIES

1. Create a private key

1.1 Actions

The purpose of the activity was to create the private key, which is achieved thanks to the BN (BIGNUM) function library, which handles numbers that go beyond the limits of computer architecture and performs mathematical operations with them (additions, subtractions, multiplications, etc.). `private_key.c` (Figure 1.1.1) calculates and prints the private key, implementing some operations. In more detail:

in lines 26 and 30 initializes two large numbers in the BN structures, `p` and `q`

```
BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
```

on line 38 initializes the integer 1 in the structure BN, `one`

```
BN_dec2bn(&one, "1");
```

in line 34 initializes the public key in the BN structure, `e`

```
BN_hex2bn(&e, "0D88C3");
```

in lines 43 and 48 it calculates the differences $p - 1$ and $q - 1$ and assigns the results to the structures BN, `p1` and `q1` respectively

```
BN_sub(p1, p, one);
BN_sub(q1, q, one);
```

in line 53 it calculates the product $(p - 1) * (q - 1)$ and assigns the result to the structure BN, `phi`, that is, it calculates the number $\phi(n)$

```
BN_mul(phi, p1, q1, ctx);
```

in line 58 it calculates the product $p \bmod N$ * q and assigns the result to the structure BN, `N`, that is, it calculates

```
BN_mul(N, p, q, ctx);
```

in line 63 it calculates the inverse of the arithmetic remainder $e * d \bmod \phi(n) = 1$ in the structure BN, `d`, that is, it calculates the private key

```
BN_mod_inverse(d, e, phi, ctx);
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

private_key.c
1  #include <stdio.h>
2  #include <openssl/bn.h>
3
4  #define NBITS 256
5
6  void printBN(char *msg, BIGNUM *a);
7
8  int main(int argc, char *argv[])
9  {
10     /*
11     * Δημιουργία των δομών τύπου BN
12     */
13     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14     BIGNUM *p = BN_new();        // Μεγάλος αριθμός p
15     BIGNUM *q = BN_new();        // Μεγάλος αριθμός q
16     BIGNUM *p1 = BN_new();       // Αφαίρεση p - 1
17     BIGNUM *q1 = BN_new();       // Αφαίρεση q - 1
18     BIGNUM *one = BN_new();      // Ακέραιος 1
19     BIGNUM *phi = BN_new();      // Αριθμός  $\phi(N) = (p - 1) * (q - 1)$ 
20     BIGNUM *N = BN_new();        // Modulo N
21     BIGNUM *e = BN_new();        // Δημόσιο κλειδί (e, N)
22     BIGNUM *d = BN_new();        // Ιδιωτικό κλειδί (d, N)
23
24     /* Αρχικοποίηση του πρώτου αριθμού p */
25     BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
26
27     /* Αρχικοποίηση του πρώτου αριθμού q */
28     BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
29
30     /* Αρχικοποίηση του δημόσιου κλειδιού e ώστε να είναι πρώτος με το  $\phi(N)$  */
31     BN_hex2bn(&e, "0D88C3");
32
33     /* Μετατροπή του ακέραιου αριθμού 1 σε δομή τύπου BN */
34     BN_dec2bn(&one, "1");
35
36     /* Υπολογισμός του πρώτου όρου του  $\phi(N)$  */
37     BN_sub(p1, p, one);
38
39     /* Υπολογισμός της αφαίρεσης q - 1 */
40     BN_sub(q1, q, one);
41
42     /* Υπολογισμός του αριθμού  $\phi(N)$  */
43     BN_mul(phi, p1, q1, ctx);
44
45     /* Υπολογισμός του modulo N */
46     BN_mul(N, p, q, ctx);
47
48     /* Υπολογισμός του αντίστροφου σε αριθμητική υπολοίπου,  $e * d \bmod \phi(N) = 1$  */
49     BN_mod_inverse(d, e, phi, ctx);
50
51     /* Εκτύπωση του modulo N = p * q */
52     printBN("N =\t", N);
53
54     /* Εκτύπωση του αριθμού  $\phi(N) = (p - 1) * (q - 1)$  */
55     printBN("Phi =\t", phi);
56
57     /* Εκτύπωση του ιδιωτικού κλειδιού d */
58     printBN("d=\t", d);
59 }

```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

76  /*
77  * Αποδέσμευση των δομών τύπου BN από την μνήμη
78  */
79  BN_CTX_free(ctx);
80  BN_free(p);
81  BN_free(q);
82  BN_free(p1);
83  BN_free(q1);
84  BN_free(one);
85  BN_free(phi);
86  BN_free(N);
87  BN_free(e);
88  BN_free(d);
89
90  return 0;
91  }
92
93  /*
94  * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
95  */
96  void printBN(char *msg, BIGNUM *a)
97  {
98      char *number_str = BN_bn2hex(a);
99      printf("%s %s\n", msg, number_str);
100     OPENSSL_free(number_str);
101 }

```

Figure 1.1.1. The private_key.c program

Then, we compiled the code with the parameter

```
-lcrypto
```

where it links to the OpenSSL BN library of large number manipulation functions.

The program compilation command is:

```
gcc private_key.c -o private_key -lcrypto
```

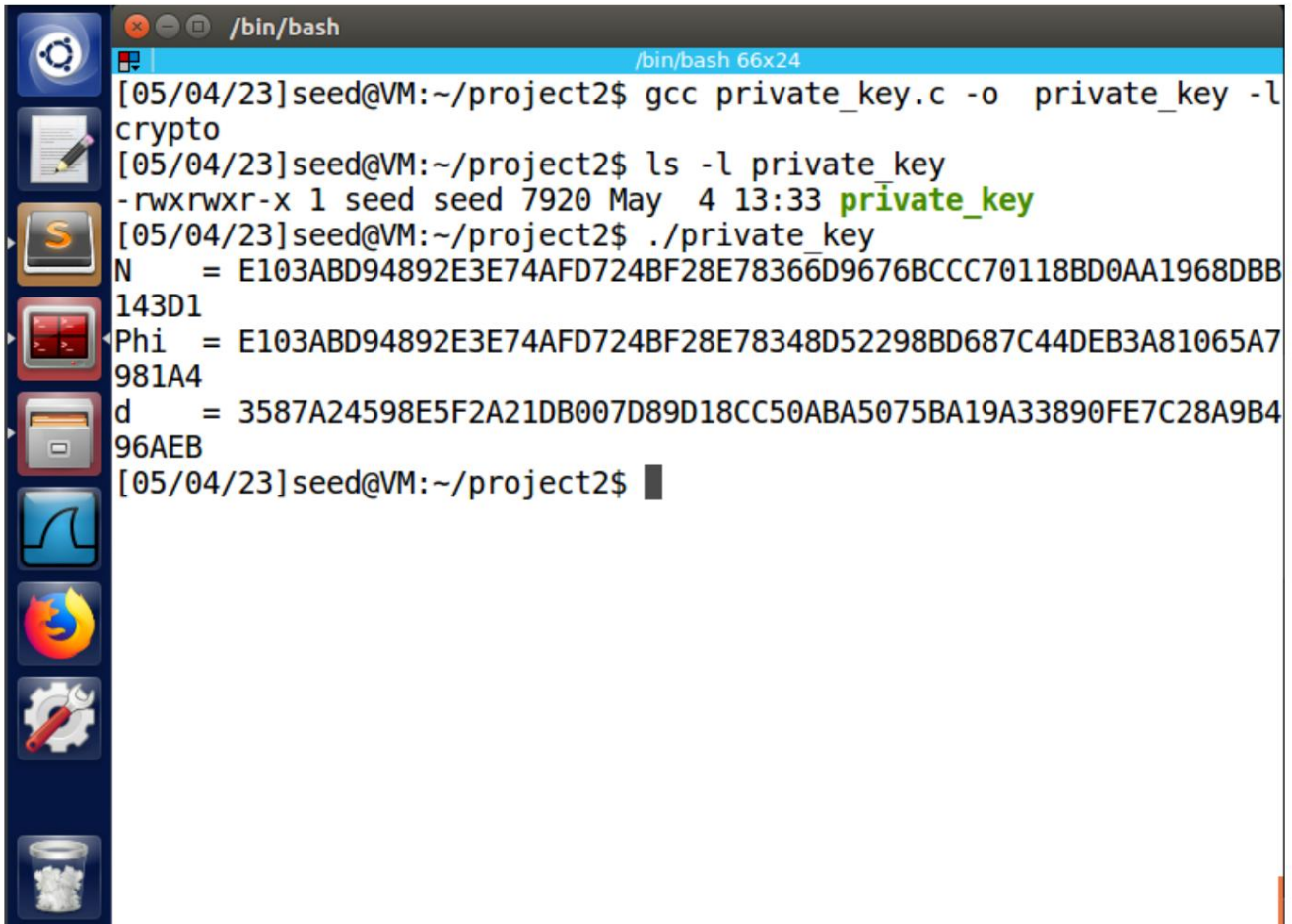
and since there were no syntax errors, the command output was empty and the executable private_key file was created.

Finally, we ran the private_key program with the command:

```
./private_key
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

1.2 Results



```

/bin/bash
[05/04/23]seed@VM:~/project2$ gcc private_key.c -o private_key -l
crypto
[05/04/23]seed@VM:~/project2$ ls -l private_key
-rwxrwxr-x 1 seed seed 7920 May  4 13:33 private_key
[05/04/23]seed@VM:~/project2$ ./private_key
N      = E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB
143D1
Phi    = E103ABD94892E3E74AFD724BF28E78348D52298BD687C44DEB3A81065A7
981A4
d      = 3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B4
96AEB
[05/04/23]seed@VM:~/project2$

```

Figure 1.2.1 Compiling and running the private_key.c program

1.3 Observations

The modulo N was calculated from the mathematical formula $N = p * q$, $\phi(\phi)$ from the formula $\phi(\phi) = (p - 1) * (q - 1)$ and the private key d from the formula $d = e * d \bmod \phi(\phi) = 1$. The public key e is a prime number with $\phi(\phi)$.

2. Message encryption

2.1 Actions

In this activity we encrypted a message thanks to the crypto_msg.c program. The message is the name of the student "Vasilis Athanasiou" and we initially encoded it in hexadecimal format with the python script

```
python -c 'print("Vasilis Athanasiou".encode("hex"))'
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

crypto_msg.c (Figure 2.1.1) encrypts this message by implementing mathematical operations with large numbers thanks to the BN function library that handles numbers that go beyond the limits of computer architecture. In more detail:

in line 24 initializes the public key in the BN structure, e

```
BN_hex2bn(&e, "0D88C3");
```

on line 29 initializes the modulo in the structure BN, N

```
BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
```

in line 34 initializes the private key in the structure BN, d

```
BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
```

in line 39 initializes the encoded message "Vasilis Athanasiou" in the structure BN, M

```
BN_hex2bn(&M, "566173696C697320417468616E6173696F75");
```

in line 44 it calculates the mathematical operation $M^e \bmod N$ and assigns the result to the structure BN, C, that is, it encrypts the message "Vasilis Athanasiou" (structure M)

```
BN_mod_exp(C, M, e, N, ctx);
```

d on line 53 calculates the mathematical operation $C^d \bmod N$ and assigns the result to the structure BN, D, that is, decrypts the message "Vasilis Athanasiou" (structure M)

```
BN_mod_exp(D, C, d, N, ctx);
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

crypto_msg.c
1  #include <stdio.h>
2  #include <openssl/bn.h>
3
4  #define NBITS 256
5
6  void printBN(char *msg, BIGNUM *a);
7
8  int main(int argc, char *argv[])
9  {
10     /*
11      * Δημιουργία των δομών τύπου BN
12      */
13     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14     BIGNUM *M = BN_new();        // Μήνυμα με το ονοματεπώνυμο του σπουδαστή
15     BIGNUM *C = BN_new();        // Κρυπτογραφημένο μήνυμα
16     BIGNUM *D = BN_new();        // Αποκρυπτογραφημένο μήνυμα
17     BIGNUM *e = BN_new();        // Δημόσιο κλειδί (e, N)
18     BIGNUM *N = BN_new();        // Modulo N
19     BIGNUM *d = BN_new();        // Ιδιωτικό κλειδί (d, N)
20
21     /*
22      * Αρχικοποίηση του δημόσιου κλειδιού (e, N) που χρησιμοποιήθηκε στην
23      * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
24      */
25     BN_hex2bn(&e, "0D88C3");
26
27     /*
28      * Αρχικοποίηση του modulo N που υπολογίστηκε στην
29      * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
30      */
31     BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
32
33     /*
34      * Αρχικοποίηση του δημόσιου κλειδιού (d, N) που υπολογίστηκε στην
35      * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
36      */
37     BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
38
39     /*
40      * Αρχικοποίηση του μηνύματος με το ονοματεπώνυμο του σπουδαστή
41      * "Vasilis Athanasiou"
42      */
43     BN_hex2bn(&M, "566173696C697320417468616E6173696F75");
44
45     /*
46      * Υπολογισμός του κρυπτογραφημένου μηνύματος με τον τύπο
47      *  $C = M^e \bmod N$ 
48      */
49     BN_mod_exp(C, M, e, N, ctx);
50
51     /*
52      * Εκτύπωση του κρυπτογραφημένου μηνύματος
53      */
54     printBN("Encrypted Message =", C);
55
56     /*
57      * Υπολογισμός του αποκρυπτογραφημένου μηνύματος με τον τύπο
58      *  $D = C^d \bmod N$ 
59      */
60     BN_mod_exp(D, C, d, N, ctx);
61
62     /*
63      * Εκτύπωση του αποκρυπτογραφημένου μηνύματος
64      */
65     printBN("Decrypted Message = ", D);
66
67     /*
68      * Αποδέσμευση των δομών τύπου BN από την μνήμη
69      */
70     BN_CTX_free(ctx);
71     BN_free(M);
72     BN_free(C);
73     BN_free(D);
74     BN_free(e);
75     BN_free(N);
76     BN_free(d);
77
78     return 0;
79 }

```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
71
72 /*
73  * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
74  */
75 void printBN(char *msg, BIGNUM *a)
76 {
77     char *number_str = BN_bn2hex(a);
78     printf("%s %s\n", msg, number_str);
79     OPENSSL_free(number_str);
80 }
```

Figure 2.1.1 The crypto_msg.c program

Then, we compiled the code with the parameter

```
-lcrypto
```

where it links to the OpenSSL BN library of large number manipulation functions.

The program compilation command is:

```
gcc crypto_msg.c -o crypto_msg -lcrypto
```

and since there were no syntax errors, the output of the command was empty and the executable file crypto_msg was created.

Then, we ran the crypto_msg program with the command:

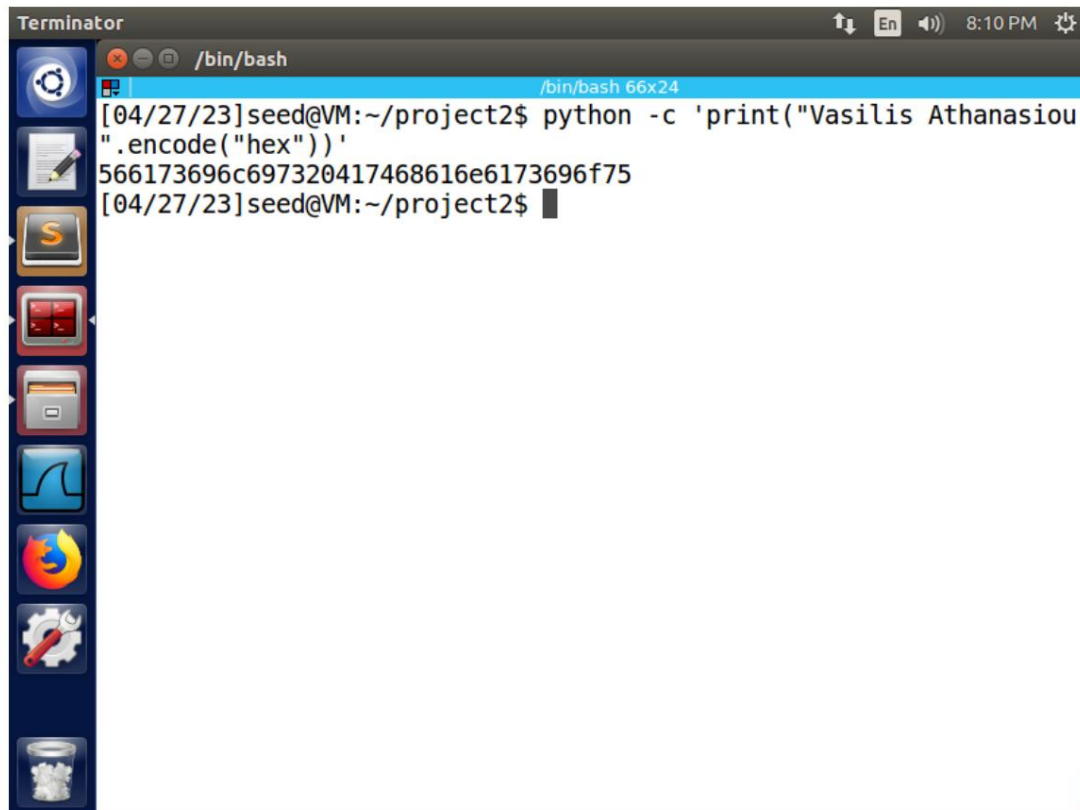
```
./crypto_msg
```

Finally, to verify that the crypto_msg.c program encrypts the message "Vasilis Athanasiou", we ran a python script, which decodes the string from hexadecimal format to human-readable format. The command is:

```
python -c 'print("566173696C697320417468616E6173696F75".decode("hex"))'
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

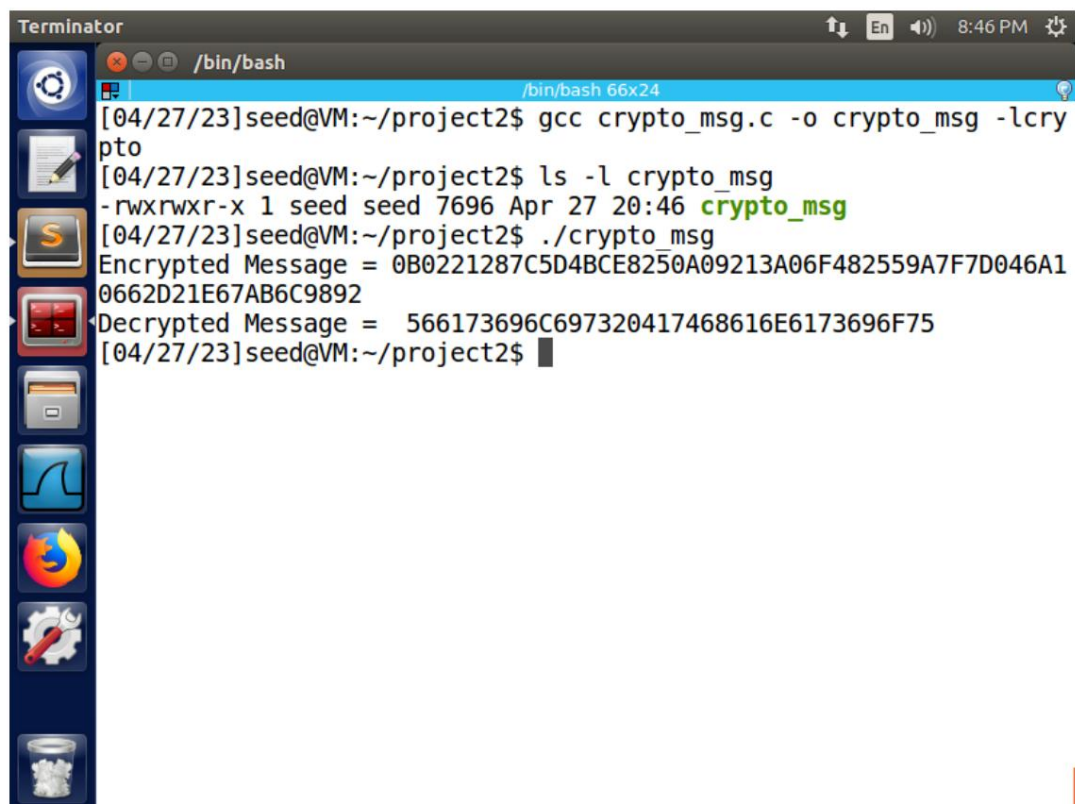
2.2 Results



The screenshot shows a Terminator terminal window with a dark background and a blue title bar. The terminal is running a bash shell. The user has entered a Python command to encode the string "Vasilis Athanasiou" into hexadecimal. The output is displayed on the next line. The terminal window has a sidebar on the left with various icons for system settings, file manager, and other applications. The top status bar shows the time as 8:10 PM.

```
Terminator /bin/bash
[04/27/23]seed@VM:~/project2$ python -c 'print("Vasilis Athanasiou".encode("hex"))'
566173696c697320417468616e6173696675
[04/27/23]seed@VM:~/project2$
```

Figure 2.2.1. Encoding of the message “Vasilis Athanasiou” in hexadecimal format



The screenshot shows a Terminator terminal window with a dark background and a blue title bar. The terminal is running a bash shell. The user has entered a series of commands to compile and run a C program named crypto_msg.c. The output shows the compilation process, the file permissions for the resulting binary, and the execution of the program which displays the encrypted and decrypted messages. The terminal window has a sidebar on the left with various icons for system settings, file manager, and other applications. The top status bar shows the time as 8:46 PM.

```
Terminator /bin/bash
[04/27/23]seed@VM:~/project2$ gcc crypto_msg.c -o crypto_msg -lcrypto
[04/27/23]seed@VM:~/project2$ ls -l crypto_msg
-rwxrwxr-x 1 seed seed 7696 Apr 27 20:46 crypto_msg
[04/27/23]seed@VM:~/project2$ ./crypto_msg
Encrypted Message = 0B0221287C5D4BCE8250A09213A06F482559A7F7D046A10662D21E67AB6C9892
Decrypted Message = 566173696C697320417468616E6173696675
[04/27/23]seed@VM:~/project2$
```

Figure 2.2.2 Compiling and running the crypto_msg.c program

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

Terminator                                     8:51 PM
/bin/bash
[04/27/23]seed@VM:~/project2$ python -c 'print("Vasilis Athanasiou".encode("hex"))'
566173696c697320417468616e6173696f75
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ ./crypto_msg
Encrypted Message = 0B0221287C5D4BCE8250A09213A06F482559A7F7D046A10662D21E67AB6C9892
Decrypted Message = 566173696C697320417468616E6173696F75
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ python -c 'print("566173696C697320417468616E6173696F75".decode("hex"))'
Vasilis Athanasiou
[04/27/23]seed@VM:~/project2$
  
```

Figure 2.2.3. Verification of successful encryption of the message “Vasilis Athanasiou”

2.3 Observations

The encryption of the message was calculated by the mathematical formula $C = M^e \bmod N$.

The decryption of the message was calculated by the mathematical formula $D = C^d \bmod N$. The python script decode the string from hexadecimal to readable form, taking as an argument the structure D calculated by the crypto_msg.c program, print the original message in the output (Vasilis Athanasiou) which we placed as an entrance to the M. structure.

3. Message decryption

3.1 Actions

In this activity we decrypted a message thanks to the decrypto_msg.c program. This message is encrypted in hexadecimal format.

$C = 7074A77B724869F59EBA8B790CEAA6A33B2E268A00BAF124BA7C3A6BDE690F36$

and the purpose of the activity was to decrypt it and then convert it into a readable format with the python script.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

decrypto_msg.c (Figure 3.1.1) decrypts this message by implementing mathematical operations with large numbers thanks to the BN function library that handles numbers that go beyond the limits of computer architecture. In more detail:

in line 23 initializes the public key in the BN structure, e

```
BN_hex2bn(&e, "0D88C3");
```

on line 28 initializes the modulo in the structure BN, N

```
BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
```

in line 33 initializes the private key in the structure BN, d

```
BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
```

on line 37 initializes the encrypted message, encoded in hexadecimal format in the structure BN, C

```
BN_hex2bn(&C, "7074A77B724869F59EBA8B790CEAA6A33B2E268A00BAF124BA7C3A6BDE690F36");
```

in line 42 it calculates the mathematical operation $C^d \bmod N$ and assigns the result to the structure BN, D, that is, it decrypts the message C which is encoded in hexadecimal format

```
BN_mod_exp(D, C, d, N, ctx);
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

1  #include <stdio.h>
2  #include <openssl/bn.h>
3
4  #define NBITS 256
5
6  void printBN(char *msg, BIGNUM *a);
7
8  int main(int argc, char *argv[])
9  {
10     /*
11     * Δημιουργία των δομών τύπου BN
12     */
13     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14     BIGNUM *C = BN_new();        // Κρυπτογραφημένο μήνυμα
15     BIGNUM *D = BN_new();        // Αποκρυπτογραφημένο μήνυμα
16     BIGNUM *e = BN_new();        // Δημόσιο κλειδί (e, N)
17     BIGNUM *N = BN_new();        // Modulo N
18     BIGNUM *d = BN_new();        // Ιδιωτικό κλειδί (d, N)
19     /*
20     * Αρχικοποίηση του δημόσιου κλειδιού (e, N) που χρησιμοποιήθηκε στην
21     * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
22     */
23     BN_hex2bn(&e, "0D88C3");
24     /*
25     * Αρχικοποίηση του modulo N που υπολογίστηκε στην
26     * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
27     */
28     BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
29     /*
30     * Αρχικοποίηση του δημόσιου κλειδιού (d, N) που υπολογίστηκε στην
31     * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
32     */
33     BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
34     /*
35     * Αρχικοποίηση του κρυπτογραφημένου μηνύματος C
36     */
37     BN_hex2bn(&C, "7074A77B724869F59EBA8B790CEAA6A33B2E268A00BAF124BA7C3A6BDE690F36");
38     /*
39     * Υπολογισμός του αποκρυπτογραφημένου μηνύματος με τον τύπο
40     *  $D = C^d \bmod N$ 
41     */
42     BN_mod_exp(D, C, d, N, ctx);
43     /*
44     * Εκτύπωση του αποκρυπτογραφημένου μηνύματος
45     */
46     printBN("Decrypted Message = ", D);
47     /*
48     * Αποδέσμευση των δομών τύπου BN από την μνήμη
49     */
50     BN_CTX_free(ctx);
51     BN_free(C);
52     BN_free(D);
53     BN_free(e);
54     BN_free(N);
55     BN_free(d);
56
57     return 0;
58 }
59
60 /*
61 * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
62 */
63 void printBN(char *msg, BIGNUM *a)
64 {
65     char *number_str = BN_bn2hex(a);
66     printf("%s %s\n", msg, number_str);
67     OPENSSL_free(number_str);
68 }

```

Figure 3.1.1 The decrypto_msg.c program

Then, we compiled the code with the parameter

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
-lcrypto
```

where it links to the OpenSSL BN library of large number manipulation functions.

The program compilation command is:

```
gcc decrypto_msg.c -o decrypto_msg -lcrypto
```

and since there were no syntax errors, the command output was empty and the executable file decrypto_msg was created.

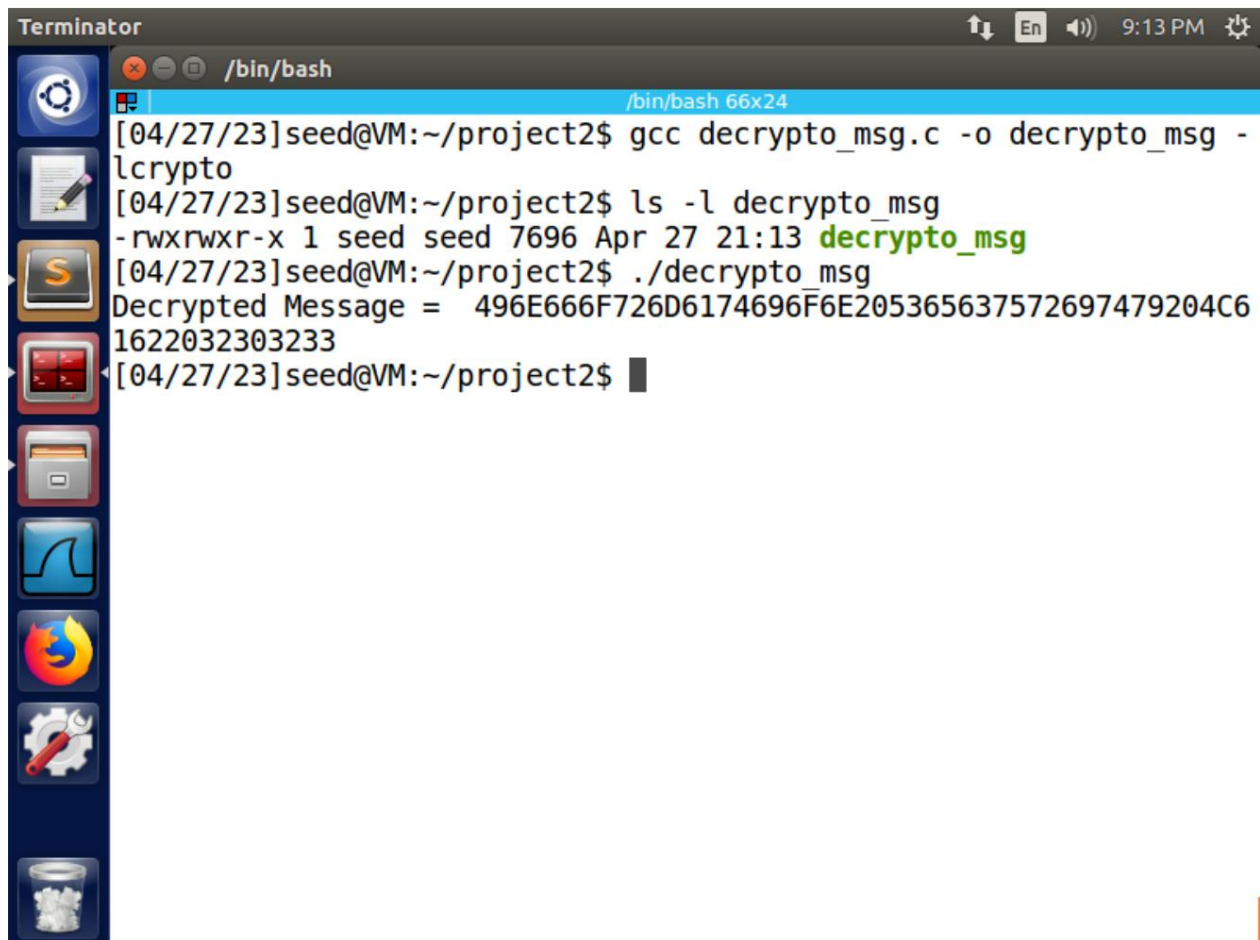
Then, we ran the decrypto_msg program with the command:

```
./decrypto_msg
```

Finally, to verify that the decrypto_msg.c program decrypts the message that is encrypted and encoded in hexadecimal format, we executed a python script, where it decodes the string from hexadecimal format to a readable format. The command is:

```
python -c  
'print("4d79204e616d65".decode("496E666F726D6174696F6E2053656375726974792  
04C61622032303233"))'
```

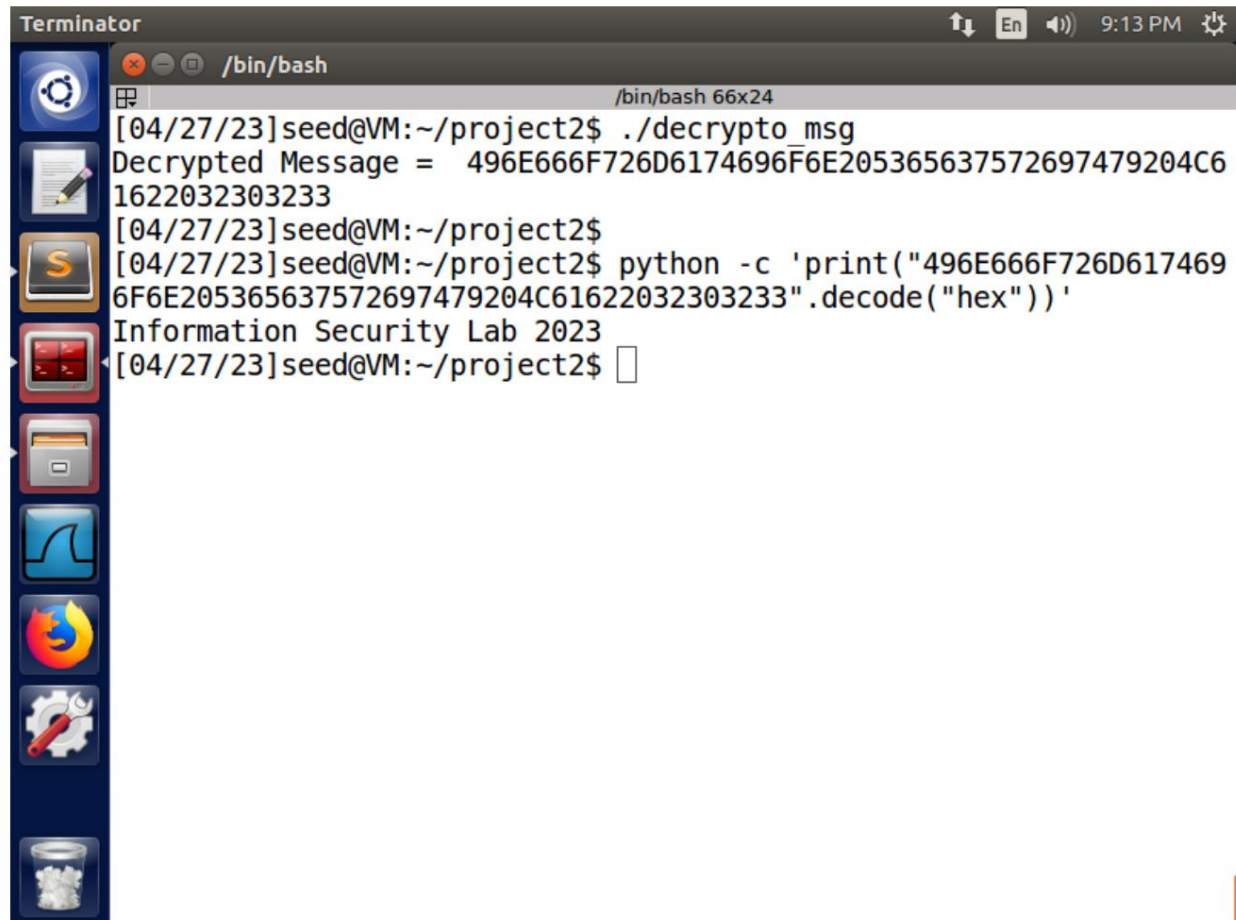
3.2 Results



```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ gcc decrypto_msg.c -o decrypto_msg -lcrypto
[04/27/23]seed@VM:~/project2$ ls -l decrypto_msg
-rwxrwxr-x 1 seed seed 7696 Apr 27 21:13 decrypto_msg
[04/27/23]seed@VM:~/project2$ ./decrypto_msg
Decrypted Message = 496E666F726D6174696F6E205365637572697479204C61622032303233
[04/27/23]seed@VM:~/project2$
```

Figure 3.2.1. Compiling and running the program decrypto_msg.c

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



The screenshot shows a Terminator terminal window with a dark background. The title bar at the top reads "Terminator" and includes standard window controls and a system clock showing 9:13 PM. The terminal prompt is `/bin/bash`. The user runs the command `./decrypto_msg`, which outputs the decrypted message: `Decrypted Message = 496E666F726D6174696F6E205365637572697479204C61622032303233`. The user then runs a Python command to decode the hex string: `python -c 'print("496E666F726D6174696F6E205365637572697479204C61622032303233".decode("hex"))'`, which outputs `Information Security Lab 2023`. The terminal window has a vertical sidebar on the left with various application icons.

```
Terminator 9:13 PM
/bin/bash
[04/27/23]seed@VM:~/project2$ ./decrypto_msg
Decrypted Message = 496E666F726D6174696F6E205365637572697479204C61622032303233
[04/27/23]seed@VM:~/project2$ python -c 'print("496E666F726D6174696F6E205365637572697479204C61622032303233".decode("hex"))'
Information Security Lab 2023
[04/27/23]seed@VM:~/project2$
```

Figure 3.2.2. Verifying message decryption

3.3 Observations

The decryption of the message was calculated by the mathematical formula $D = C \pmod N$. The message "Information Security Lab 2023" verifies the successful decryption of the message.

4. Message signature

4.1 Actions

In this activity we created the digital signature of a message thanks to the program `sign_msg.c`. The message is "Hello World" and we initially encoded it in hexadecimal format with the appropriate python script

`sign_msg.c` (Figure 4.1.1) creates the digital signature of the message by implementing mathematical operations with large numbers thanks to the BN function library that handles numbers that go beyond the limits of computer architecture. In more detail:

in line 24 initializes the public key in the BN structure, e

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
BN_hex2bn(&e, "0D88C3");
```

on line 29 initializes the modulo in the structure BN, N

```
BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB  
143D1");
```

in line 34 initializes the private key in the structure BN, d

```
BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B4  
96AEB");
```

on line 38 initializes the encrypted message, encoded in hexadecimal format in the structure BN, M1

```
BN_hex2bn(&M1, "48616C6C6F20576F726C64");
```

in line 43 it calculates the mathematical operation $M1d \bmod N$ and assigns the result to the structure BN, M2, that is, it calculates the digital signature of the message

```
BN_mod_exp(S, M1, d, N, ctx);
```

in line 48 it calculates the mathematical operation $S^e \bmod N$ and assigns the result to the structure BN, M2, that is, it decrypts the digital signature of the message that is encoded in hexadecimal format

```
BN_mod_exp(M2, S, e, N, ctx);
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
bn_sample.c x private_key.c x crypto_msg.c x decrypto_msg.c x sign_msg.c x
1 |#include <stdio.h>
2 |#include <openssl/bn.h>
3
4 |#define NBITS 256
5
6 |void printBN(char *msg, BIGNUM *a);
7
8 |int main(int argc, char *argv[])
9 |{
10 |/*
11 | * Δημιουργία των δομών τύπου BN
12 | */
13 |    BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14 |    BIGNUM *M1 = BN_new(); // Μήνυμα
15 |    BIGNUM *S = BN_new(); // Υπογραφή μηνύματος
16 |    BIGNUM *M2 = BN_new(); // Επαλήθευση υπογραφής
17 |    BIGNUM *e = BN_new(); // Δημόσιο κλειδί (e, N)
18 |    BIGNUM *N = BN_new(); // Modulo N
19 |    BIGNUM *d = BN_new(); // Ιδιωτικό κλειδί (d, N)
20 |/*
21 | * Αρχικοποίηση του δημόσιου κλειδιού (e, N) που χρησιμοποιήθηκε στην
22 | * δραστηριότητα 3 : Αποκρυπτογράφηση μηνύματος
23 | */
24 |    BN_hex2bn(&e, "0D88C3");
25 |/*
26 | * Αρχικοποίηση του modulo N που υπολογίστηκε στην
27 | * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
28 | */
29 |    BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
30 |/*
31 | * Αρχικοποίηση του δημόσιου κλειδιού (d, N) που υπολογίστηκε στην
32 | * δραστηριότητα 3 : Αποκρυπτογράφηση μηνύματος
33 | */
34 |    BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
35 |/*
36 | * Αρχικοποίηση του μηνύματος "Hello World"
37 | */
38 |    BN_hex2bn(&M1, "48616C6C6C6F20576F726C64");
39 |/*
40 | * Υπολογισμός της ψηφιακής υπογραφής του μηνύματος με τον τύπο
41 | *  $S = M1^d \mod N$ 
42 | */
43 |    BN_mod_exp(S, M1, d, N, ctx);
44 |/*
45 | * Υπολογισμός της "αποκρυπτογράφησης" της ψηφιακής υπογραφής του μηνύματος
46 | * με τον τύπο  $M2 = S^e \mod N$ 
47 | */
48 |    BN_mod_exp(M2, S, e, N, ctx);
49 |/*
50 | * Εκτύπωση της ψηφιακής υπογραφής του μηνύματος
51 | */
52 |    printBN("Message's Signature = ", S);
53 |/*
54 | * Εκτύπωση του μηνύματος
55 | */
56 |    printBN("Message \t = ", M1);
57 |/*
58 | * Εκτύπωση της "αποκρυπτογραφημένης" ψηφιακής υπογραφής του μηνύματος
59 | */
60 |    printBN("Verified Signature = ", M2);
61 |/*
62 | * Αποδέσμευση των δομών τύπου BN από την μνήμη
63 | */
64 |    BN_CTX_free(ctx);
65 |    BN_free(M1);
66 |    BN_free(S);
67 |    BN_free(M2);
68 |    BN_free(e);
69 |    BN_free(N);
70 |    BN_free(d);
71 |
72 |    return 0;
73 |}
74
75 |/*
76 | * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
77 | */
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
78 void printBN(char *msg, BIGNUM *a)
79 {
80     char *number_str = BN_bn2hex(a);
81     printf("%s %s\n", msg, number_str);
82     OPENSSL_free(number_str);
83 }
```

Figure 4.1.1. The sign_msg.c program

Then, we compiled the code with the parameter

```
-lcrypto
```

where it links to the OpenSSL BN library of large number manipulation functions.

The program compilation command is:

```
gcc sign_msg.c -o sign_msg -lcrypto
```

and since there were no syntax errors, the command output was empty and the executable file sign_msg was created.

Then, we ran the sign_msg program with the command:

```
./sign_msg
```

Finally, to verify that the sign_msg.c program decrypts the digital signature of the message that is encrypted and encoded in hexadecimal format, we ran a python script, which decodes the string from hexadecimal format to human-readable format.

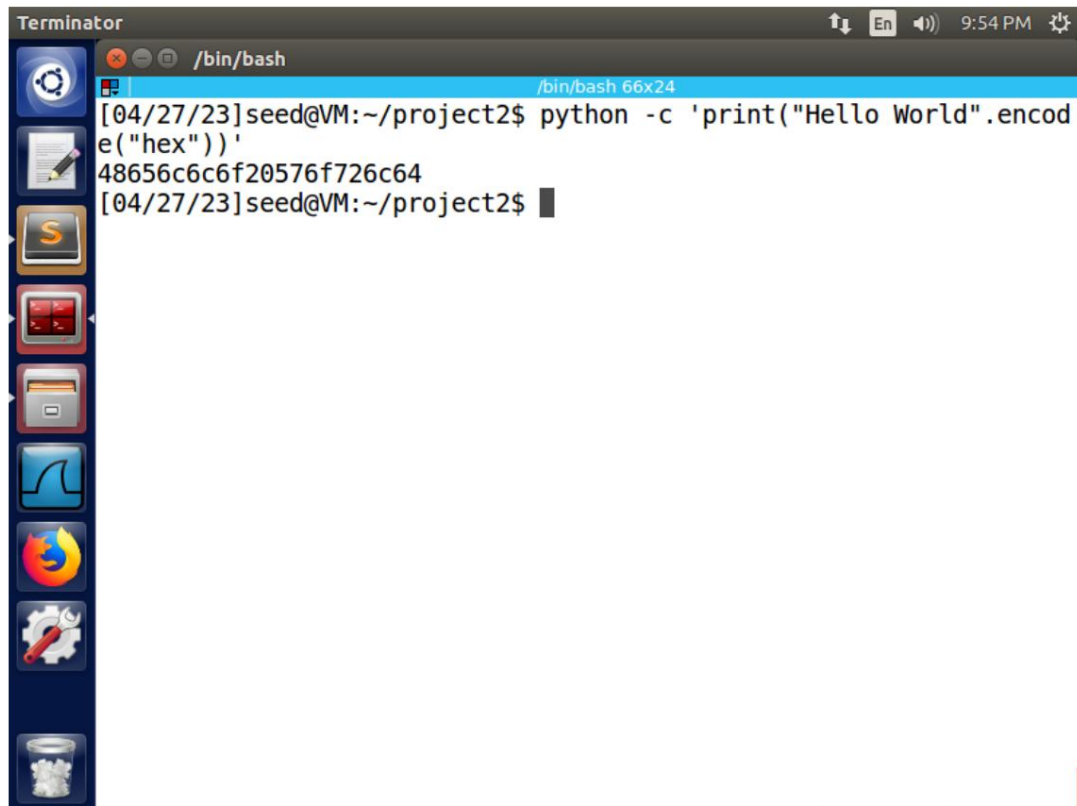
Then, we changed the message by one letter to "Hello World" and re-encoded it in hexadecimal format.

Then, we ran the sign_msg program (without changing d, e and N) with the command:

```
./sign_msg
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

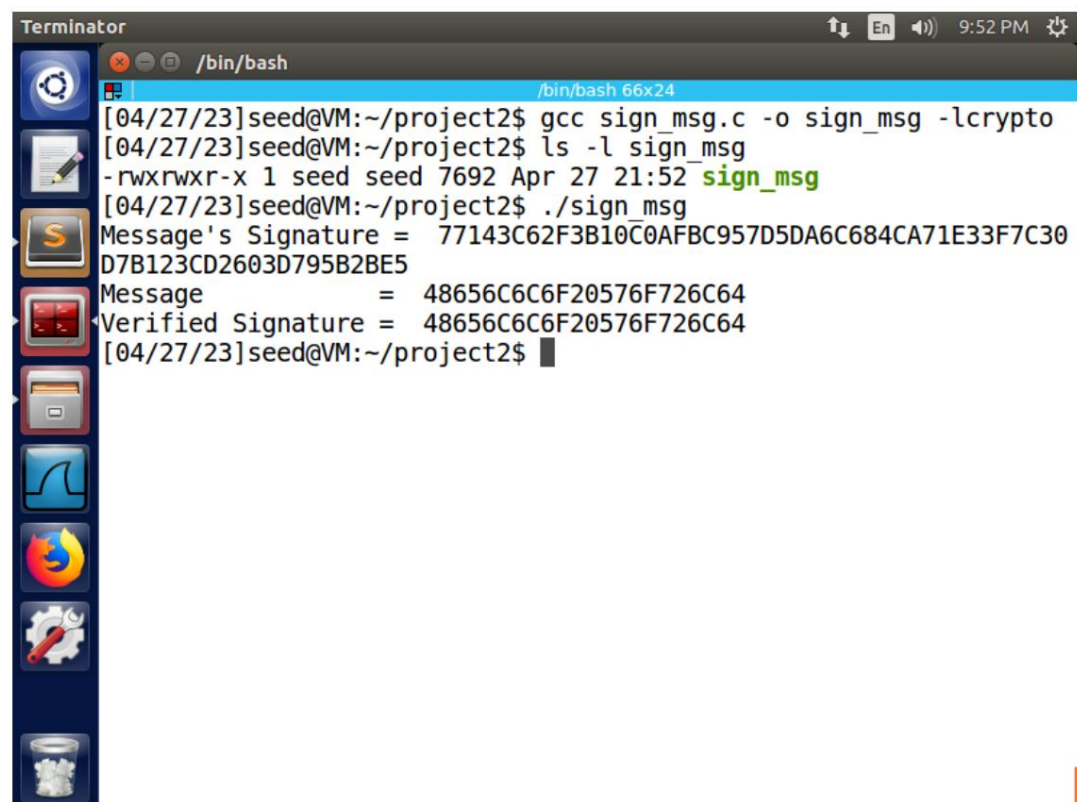
4.2 Results



The screenshot shows a Terminator terminal window with a dark theme. The title bar includes the text 'Terminator' and system icons for volume, network, and time (9:54 PM). The terminal window has a title bar with '/bin/bash' and a tab labeled '/bin/bash 66x24'. The terminal content shows a user prompt '[04/27/23]seed@VM:~/project2\$' followed by the command 'python -c \'print("Hello World").encode("hex")\''. The output is '48656c6c6f20576f726c64'. The prompt is then '[04/27/23]seed@VM:~/project2\$'.

```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Hello World").encode("hex")'
48656c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
```

Figure 4.2.1. Encoding the message “Hello World” in hexadecimal format

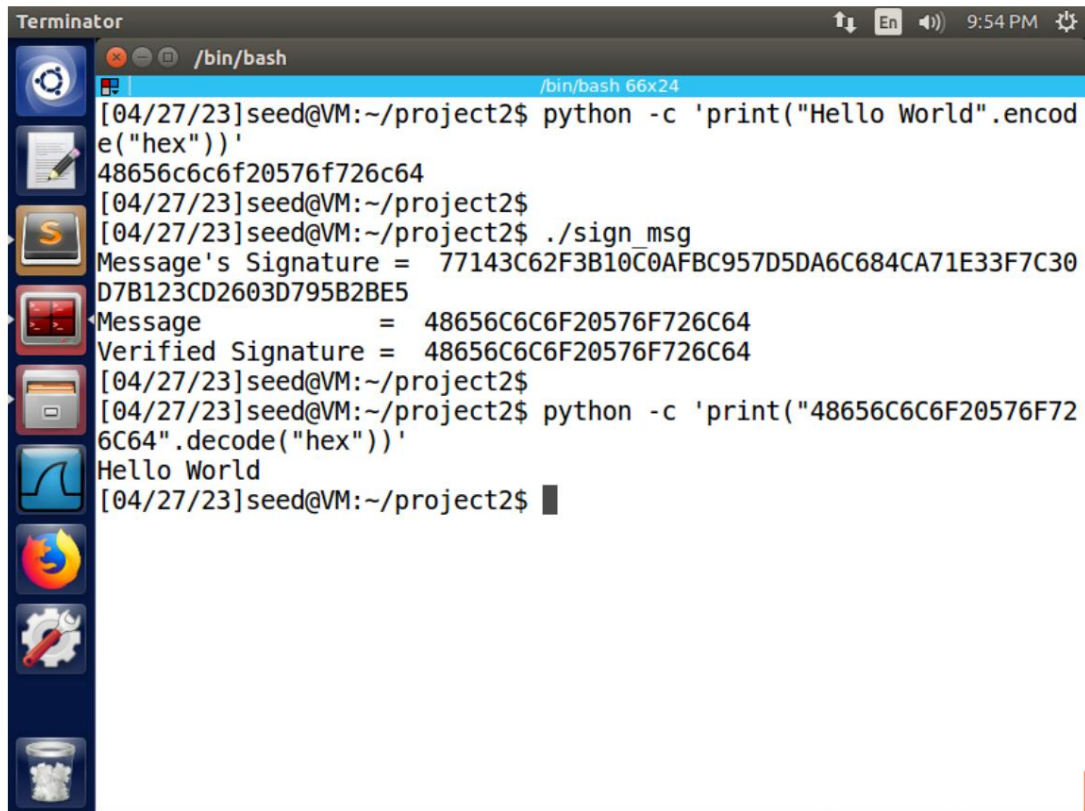


The screenshot shows a Terminator terminal window with a dark theme. The title bar includes the text 'Terminator' and system icons for volume, network, and time (9:52 PM). The terminal window has a title bar with '/bin/bash' and a tab labeled '/bin/bash 66x24'. The terminal content shows a user prompt '[04/27/23]seed@VM:~/project2\$' followed by the command 'gcc sign_msg.c -o sign_msg -lcrypto'. The next prompt is '[04/27/23]seed@VM:~/project2\$' followed by 'ls -l sign_msg', which outputs '-rwxrwxr-x 1 seed seed 7692 Apr 27 21:52 sign_msg'. The next prompt is '[04/27/23]seed@VM:~/project2\$' followed by './sign_msg'. The output is 'Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30D7B123CD2603D795B2BE5'. The next line is 'Message = 48656C6C6F20576F726C64'. The next line is 'Verified Signature = 48656C6C6F20576F726C64'. The final prompt is '[04/27/23]seed@VM:~/project2\$'.

```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ gcc sign_msg.c -o sign_msg -lcrypto
[04/27/23]seed@VM:~/project2$ ls -l sign_msg
-rwxrwxr-x 1 seed seed 7692 Apr 27 21:52 sign_msg
[04/27/23]seed@VM:~/project2$ ./sign_msg
Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30D7B123CD2603D795B2BE5
Message = 48656C6C6F20576F726C64
Verified Signature = 48656C6C6F20576F726C64
[04/27/23]seed@VM:~/project2$
```

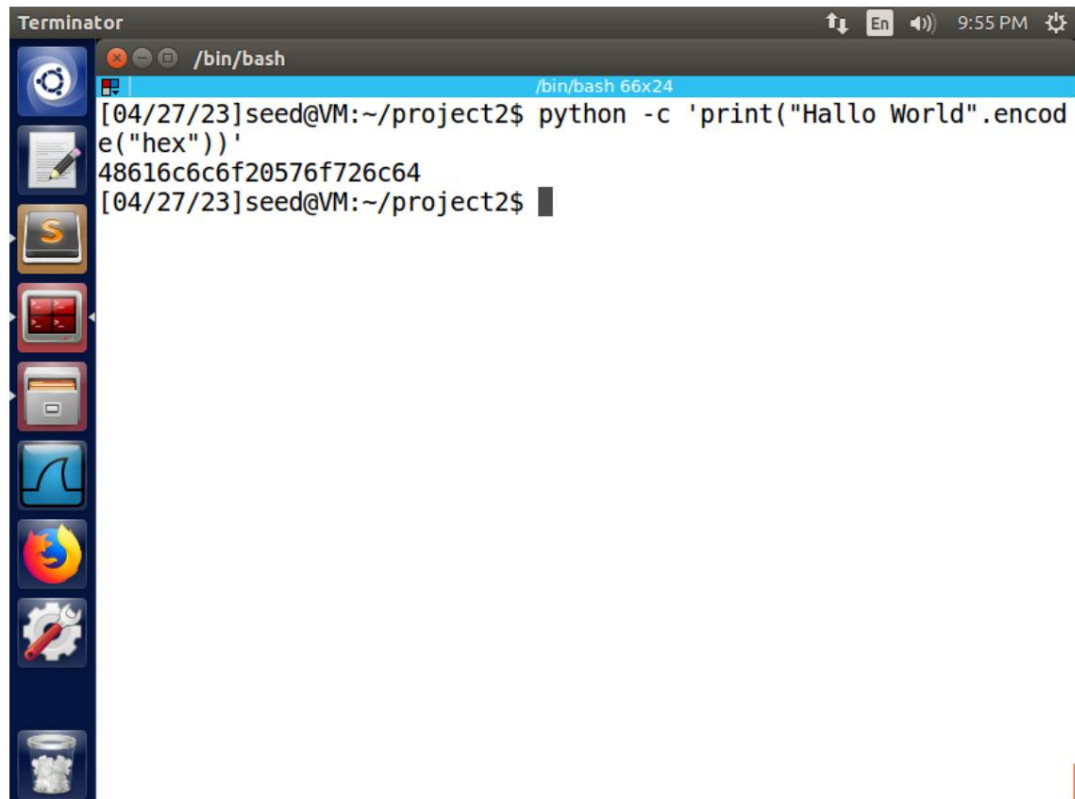
Figure 4.2.2. Compiling and running the program sign_msg.c

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Hello World".encode("hex"))'
48656c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ ./sign_msg
Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30
D7B123CD2603D795B2BE5
Message = 48656c6c6f20576f726c64
Verified Signature = 48656c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ python -c 'print("48656c6c6f20576f726c64".decode("hex"))'
Hello World
[04/27/23]seed@VM:~/project2$
```

Figure 4.2.3. Verifying the decryption of the digital signature of the message "Hello World"



```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Hallo World".encode("hex"))'
48616c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
```

Figure 4.2.4. Encoding the message "Hello World" in hexadecimal format

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

Terminator
/bin/bash
[04/27/23]seed@VM:~/project2$ python -c 'print("Halo World".encode("hex"))'
48616c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$ ./sign_msg
Message's Signature = 09EF86E90C92C9AA909162367C0A3838C3B6A1908DB
9AC73A8C9D483A0796236
Message = 48616c6c6f20576f726c64
Verified Signature = 48616c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ python -c 'print("48616c6c6f20576f72
6c64".decode("hex"))'
Halo World
[04/27/23]seed@VM:~/project2$

```

Figure 4.2.5. Verifying the decryption of the digital signature of the message "Hello World"

4.3 Observations

The digital signatures of the messages were calculated by the mathematical formula $S = M1d \bmod N$.
The decryptions of the digital signatures were calculated by the mathematical formula $M2 = S \bmod N$.

```

Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30
D7B123CD2603D795B2BE5

```

Figure 4.3.1. The digital signature of the message "Hello World"

```

Message's Signature = 09EF86E90C92C9AA909162367C0A3838C3B6A1908DB
9AC73A8C9D483A0796236

```

Figure 4.3.2. The digital signature of the message "Hello World"

Even though the difference between the two messages is one letter (one bit), their signatures are different, due to the digital signature calculation formula $S = M1d \bmod N$. The $M1$ changes very little in opposite to the force $M1d$ which changes a lot.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

5. Signature Verification

5.1 Actions

In this activity we verified the digital signature of a message thanks to the verify_sign_msg.c program.

Case A

The message is "Launch a missile" and we initially encoded it in hexadecimal format with the appropriate python script. The sender is Bob and the receiver is Alice.

verify_sign_msg.c (Figure 5.1.1) verifies the digital signature of the message by implementing mathematical operations with large numbers thanks to the BN function library that handles numbers that go beyond the limits of the computer architecture. In more detail:

in line 23 initializes the public key in the BN structure, e

```
BN_hex2bn(&e, "010001");
```

on line 27 initializes the modulo in the structure BN, N

```
BN_hex2bn(&N, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F1816115");
```

on line 31 initializes the digital signature of the message in the structure BN, S

```
BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F");
```

on line 35 initializes the encrypted message, encoded in hexadecimal format in the structure BN, M1

```
BN_hex2bn(&M1, "4C61756E63682061206D697373696C652E");
```

in line 40 it calculates the mathematical operation $S^e \bmod N$ and assigns the result to the structure BN, M2, that is, it calculates the decryption of the digital signature of the message

```
BN_mod_exp(M2, S, e, N, ctx);
```

in lines 56-64 compares the message with the message signature verification

```
areEqual = BN_cmp(M1, M2);

switch (areEqual)
{
    case 0 :

        printf("Message and Verified Signature are equal.\n");
        break;

    default :

        printf("Message and Verified Signature are NOT equal.\n");
}
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

br private_key.c x crypto_msg.c x decrypto_msg.c x sign_msg.c x verify_sign.c x
1 #include <stdio.h>
2 #include <openssl/bn.h>
3
4 #define NBITS 256
5
6 void printBN(char *msg, BIGNUM *a);
7
8 int main(int argc, char *argv[])
9 {
10     int areEqual;
11     /*
12      * Δημιουργία των δομών τύπου BN
13      */
14     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
15     BIGNUM *M1 = BN_new(); // Μήνυμα
16     BIGNUM *M2 = BN_new(); // Επαλήθευση υπογραφής
17     BIGNUM *S = BN_new(); // Υπογραφή μηνύματος
18     BIGNUM *e = BN_new(); // Δημόσιο κλειδί (e, N)
19     BIGNUM *N = BN_new(); // Modulo N
20     /*
21      * Αρχικοποίηση του δημόσιου κλειδιού (e, N)
22      */
23     BN_hex2bn(&e, "010001");
24     /*
25      * Αρχικοποίηση του modulo N
26      */
27     BN_hex2bn(&N, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
28     /*
29      * Αρχικοποίηση της υπογραφής του μηνύματος
30      */
31     BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F");
32     /*
33      * Αρχικοποίηση του μηνύματος "Launch a missile."
34      */
35     BN_hex2bn(&M1, "4C61756E63682061206D6973737696C652E");
36     /*
37      * Υπολογισμός της "αποκρυπτογράφησης" της ψηφιακής υπογραφής του μηνύματος
38      * με τον τύπο  $M2 = S^e \bmod N$ 
39      */
40     BN_mod_exp(M2, S, e, N, ctx);
41     /*
42      * Εκτύπωση της ψηφιακής υπογραφής του μηνύματος
43      */
44     printBN("Message's Signature =", S);
45     /*
46      * Εκτύπωση του μηνύματος
47      */
48     printBN("Message \t = ", M1);
49     /*
50      * Εκτύπωση της "αποκρυπτογραφημένης" ψηφιακής υπογραφής του μηνύματος
51      */
52     printBN("Verified Signature = ", M2);
53     /*
54      * Σύγκριση του μηνύματος με την επαλήθευση υπογραφής
55      */
56     areEqual = BN_cmp(M1, M2);
57     switch (areEqual)
58     {
59         case 0 :
60             printf("Message and Verified Signature are equal.\n");
61             break;
62         default :
63             printf("Message and Verified Signature are NOT equal.\n");
64     }
65     /*
66      * Αποδέσμευση των δομών τύπου BN από την μνήμη
67      */
68     BN_CTX_free(ctx);
69     BN_free(M1);
70     BN_free(S);
71     BN_free(M2);
72     BN_free(e);
73     BN_free(N);
74

```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

75     return 0;
76 }
77
78 /*
79  * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
80  */
81 void printBN(char *msg, BIGNUM *a)
82 {
83     char *number_str = BN_bn2hex(a);
84     printf("%s %s\n", msg, number_str);
85     OPENSSL_free(number_str);
86 }

```

Figure 5.1.1. The verify_sign_msg.c program

Then, we compiled the code with the parameter

```
-lcrypto
```

where it links to the OpenSSL BN library of large number manipulation functions.

The program compilation command is:

```
gcc verify_sign_msg.c -o verify_sign_msg -lcrypto
```

and since there were no syntax errors, the command output was empty and the executable file verify_sign_msg was created.

Then, we ran the verify_sign_msg program with the command:

```
./verify_sign_msg
```

Finally, to verify that the verify_sign_msg.c program decrypts the digital signature of Bob's message, which is encrypted and encoded in hexadecimal format, we ran a python script, which decodes the string from hexadecimal format to human-readable format. The command is:

Next, we altered the digital signature of the message (Figure 5.1.2) by one bit (from 2F to 3F) at the end) and we repeated the activity

We ran the verify_sign_msg program (with the same other values e, N, M1) with the command:

```
./verify_sign_msg
```

```

19
20 Αρχικοποίηση του δημόσιου κλειδιού (e, N)
21
22 BN_hex2bn(&e, "010001");
23
24 Αρχικοποίηση του modulo N
25
26 BN_hex2bn(&N, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
27
28 Αρχικοποίηση της υπογραφής του μηνύματος
29
30 BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6863F");
31
32 Αρχικοποίηση του μηνύματος "Launch a missile."
33
34 BN_hex2bn(&M1, "4C61756E63682061206D697373696C652E");
35

```

Figure 5.1.2 Altering the digital signature of Bob's message

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Case B

In this case, Alice sends Bob the message "Please transfer me \$2000.Alice." and we repeated the process (signature verification) with differences in the values of e (public key), N (modulo) and S (digital signature) which are depicted in Figure 5.1.3.

```

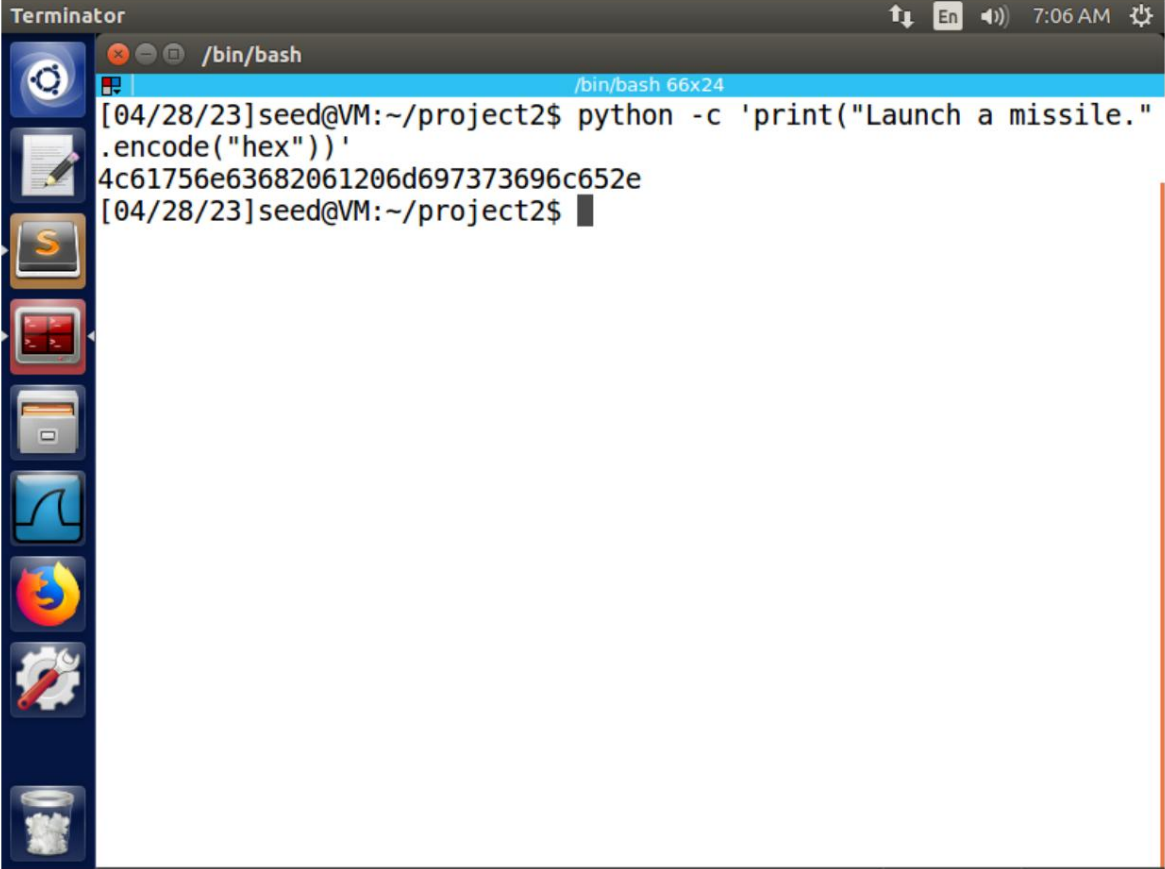
21 Αρχικοποίηση του δημόσιου κλειδιού (e, N)
22 BN_hex2bn(&e, "010001");
23
24 Αρχικοποίηση του modulo N
25 BN_hex2bn(&N, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
26
27 Αρχικοποίηση της υπογραφής του μηνύματος
28 BN_hex2bn(&S, "DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA6EDF3E95E9A7C6C7A320");
29
30 Αρχικοποίηση του μηνύματος "Please transfer me $2000.Alice."
31 BN_hex2bn(&M1, "506C65617365207472616E73666572206D6520243230302E416C6963652E");
32
33
34
35
36

```

Figure 5.1.3. The values for verifying the signature of Alice's message

5.2 Results

Case A



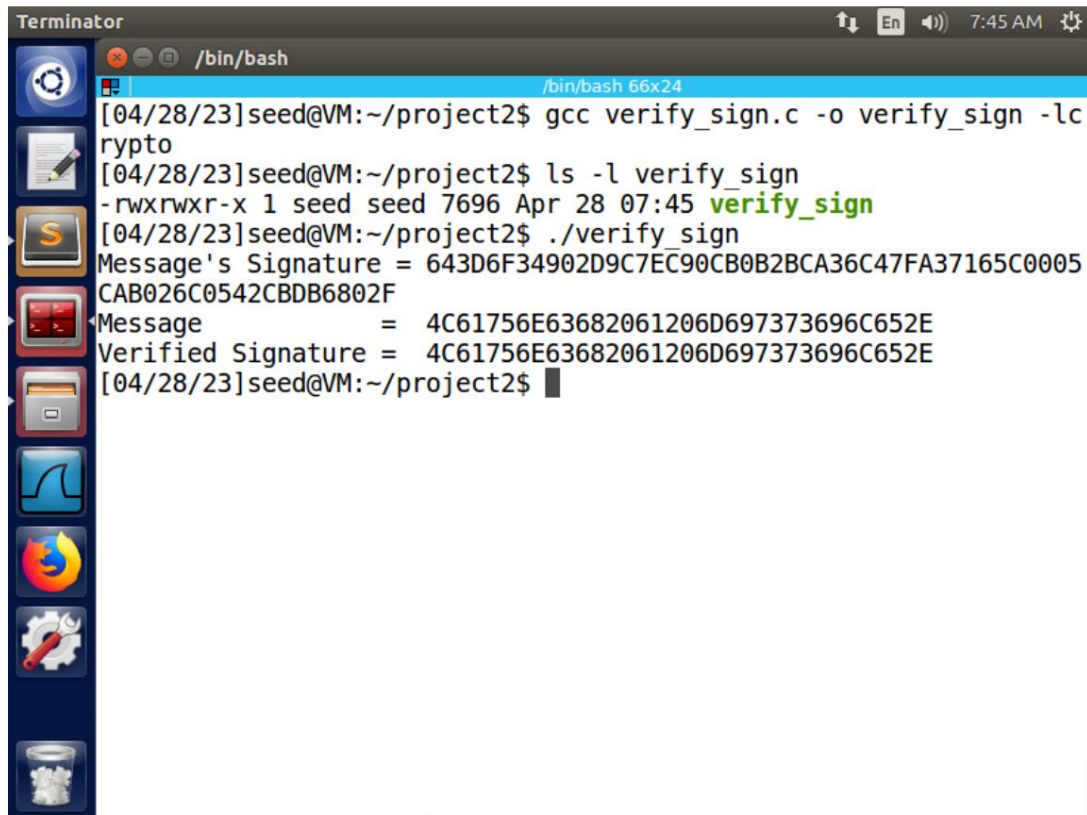
```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Launch a missile."
.encode("hex"))'
4c61756e63682061206d697373696c652e
[04/28/23]seed@VM:~/project2$

```

Figure 5.2.1. Encoding of the message "Launch a missile." in hexadecimal format

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

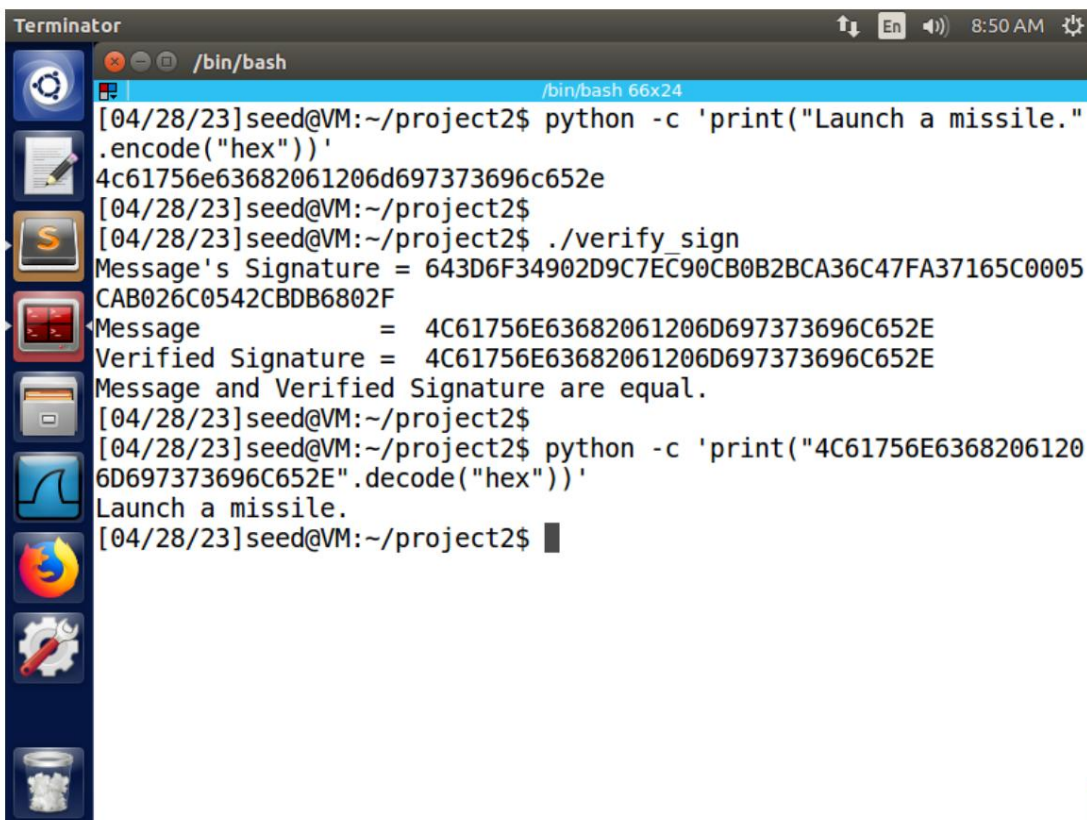


```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ gcc verify_sign.c -o verify_sign -lc
crypto
[04/28/23]seed@VM:~/project2$ ls -l verify_sign
-rwxrwxr-x 1 seed seed 7696 Apr 28 07:45 verify_sign
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005
CAB026C0542CBDB6802F
Message = 4C61756E63682061206D697373696C652E
Verified Signature = 4C61756E63682061206D697373696C652E
[04/28/23]seed@VM:~/project2$

```

Figure 5.2.2. Compiling and running the program `verify_sign_msg.c` with Bob's message



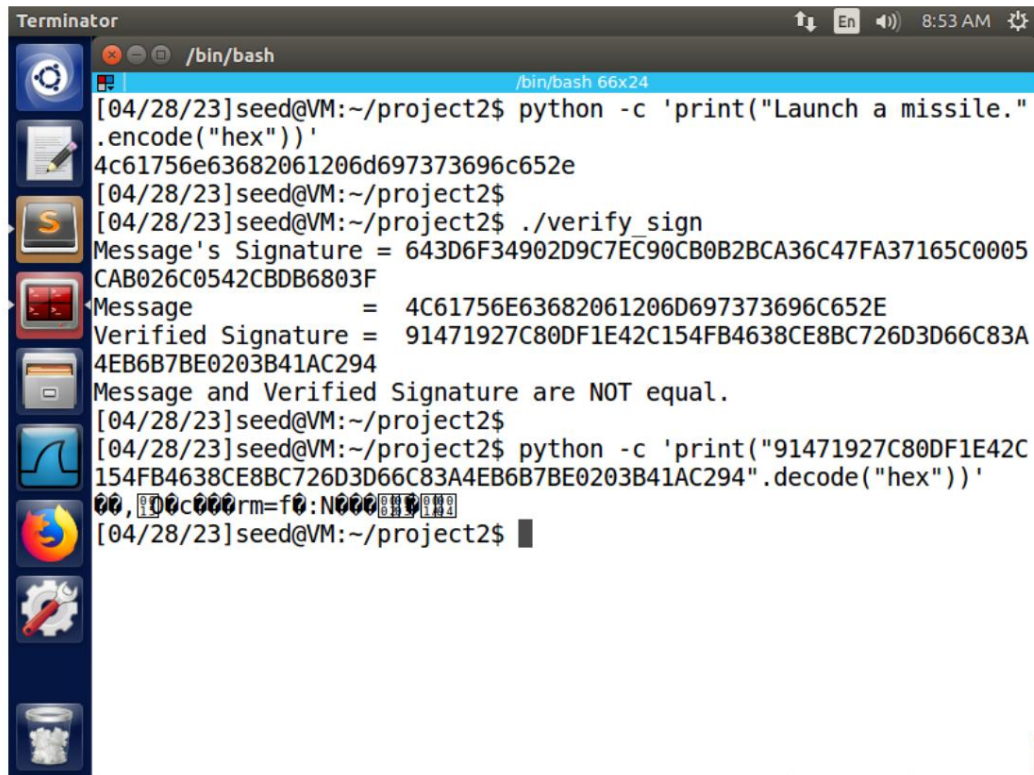
```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Launch a missile."
.encode("hex"))'
4c61756e63682061206d697373696c652e
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005
CAB026C0542CBDB6802F
Message = 4C61756E63682061206D697373696C652E
Verified Signature = 4C61756E63682061206D697373696C652E
Message and Verified Signature are equal.
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ python -c 'print("4C61756E6368206120
6D697373696C652E".decode("hex"))'
Launch a missile.
[04/28/23]seed@VM:~/project2$

```

Figure 5.2.3. Verifying the decryption of the digital signature of the message
"Launch a missile."

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



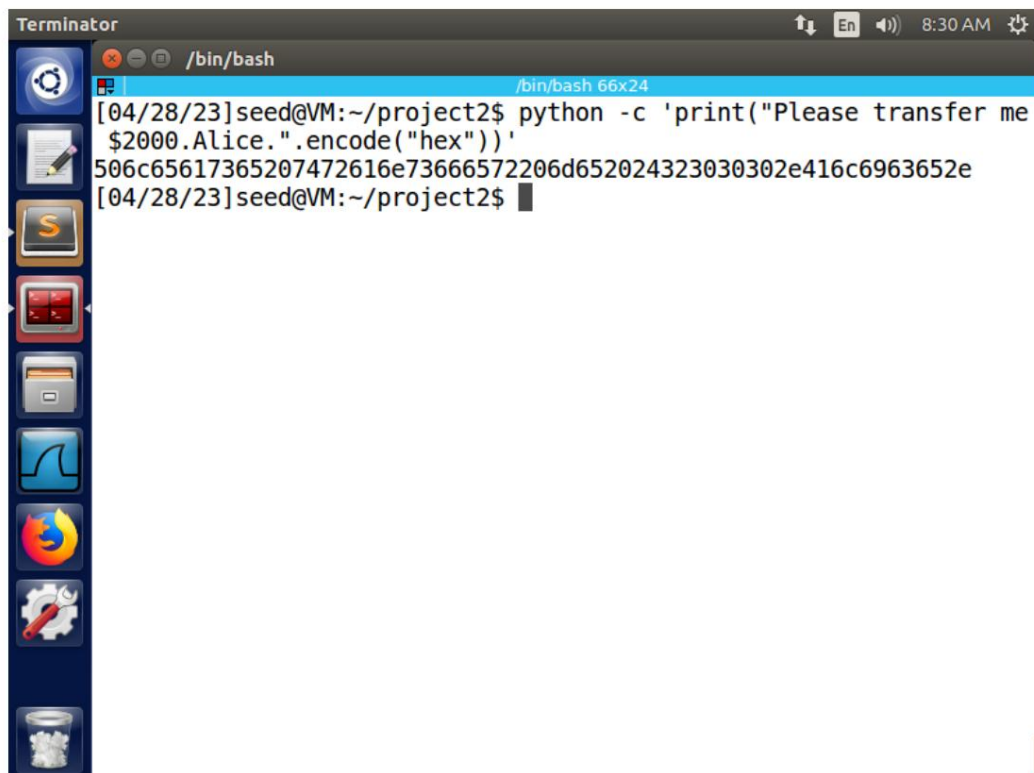
```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Launch a missile."
.encode("hex"))'
4c61756e63682061206d697373696c652e
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005
CAB026C0542CBDB6803F
Message = 4C61756E63682061206D697373696C652E
Verified Signature = 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A
4EB6B7BE0203B41AC294
Message and Verified Signature are NOT equal.
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ python -c 'print("91471927C80DF1E42C
154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294".decode("hex"))'
91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
[04/28/23]seed@VM:~/project2$

```

Figure 5.2.3. Decryption of the altered digital signature of the message "Launch a missile."

Case B



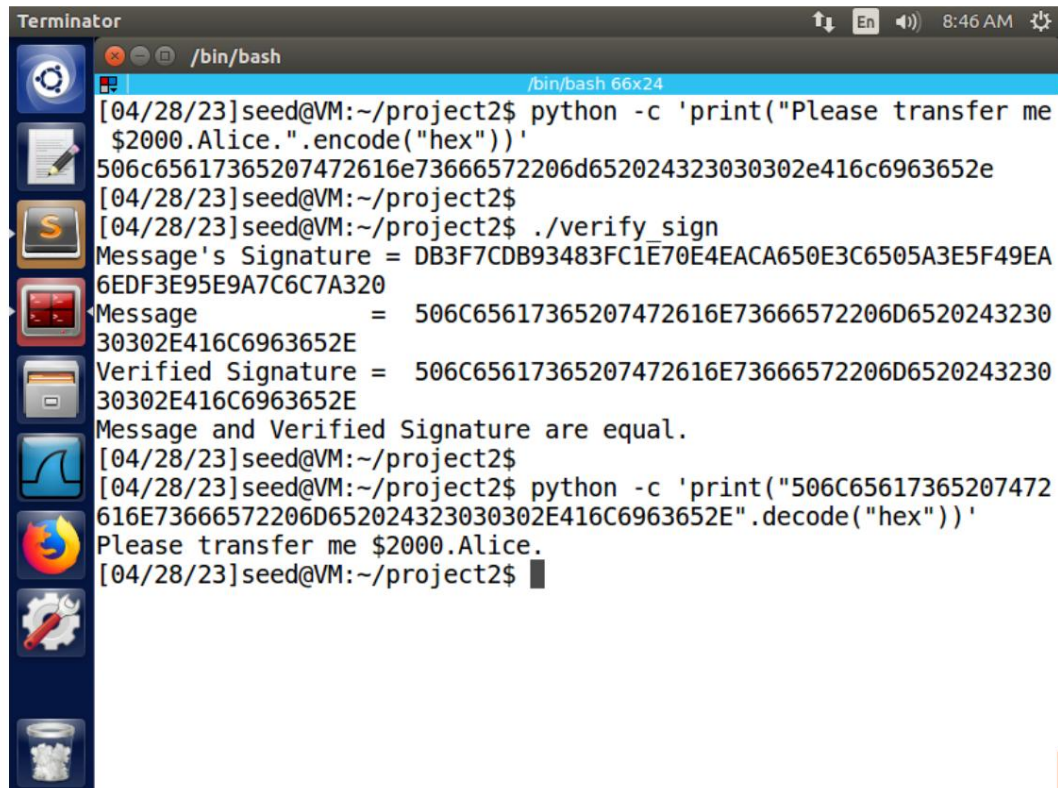
```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Please transfer me
$2000.Alice.".encode("hex"))'
506c65617365207472616e73666572206d652024323030302e416c6963652e
[04/28/23]seed@VM:~/project2$

```

Figure 5.2.4. Encoding the message "Please transfer me \$2000.Alice." in hexadecimal format

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```

Terminator
/bin/bash
[04/28/23]seed@VM:~/project2$ python -c 'print("Please transfer me
$2000.Alice.".encode("hex"))'
506c65617365207472616e73666572206d652024323030302e416c6963652e
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA
6EDF3E95E9A7C6C7A320
Message = 506C65617365207472616E73666572206D6520243230
30302E416C6963652E
Verified Signature = 506C65617365207472616E73666572206D6520243230
30302E416C6963652E
Message and Verified Signature are equal.
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ python -c 'print("506C65617365207472
616E73666572206D652024323030302E416C6963652E".decode("hex"))'
Please transfer me $2000.Alice.
[04/28/23]seed@VM:~/project2$
  
```

Figure 5.2.5. Verifying the decryption of the digital signature of the message
"Please transfer me \$2000.Alice."

5.3 Observations

The decryptions of the digital signatures were calculated by the mathematical formula $M2 = \text{mod } N$. In S^e case A of Bob's message (Launch a missile.), where we altered the digital signature of the message by one bit, we noticed during the verification of the signature that the message was not the original one. This proves that the digital signatures of the messages are unique.

6. Manual verification of X.509 certificate

6.1 Actions

In this activity we verified the digital signature of an X.509 certificate thanks to the program `verify_sign_cert.c`. We selected the web server `apachefriends.org` and to verify the signature we performed the following steps.

Step 1

- Download certificates with the command

```
openssl s_client -connect www.apachefriends.org:443 -showcerts
```

- Save the server certificate (certificate 0) in the `c0.pem` file (Figure 6.1.1)
- Save the issuer certificate (certificate 1) in the file `c1.pem` (Figure 6.1.2)

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

[illegible]

Figure 6.1.1. Server certificate

```
~/project2/c1.pem - Sublime Text (UNREGISTERED)  ↑ [En] 6:40 PM  [Settings] [Window] [File] [Edit] [Tools] [View] [Help]

c0.pem x c1.pem x signature x public_key x verify_sign_cert.c x

1 -----BEGIN CERTIFICATE-----
2 MIIEvjCCA6agAwIBAgIQBtjZBNVYQ0b2i1+nVCJ+xDANBgkqhkiG9w0BAQsFADBo
3 MQswCQYDVQQGEwJVUzEVMBMGAUEChMMRGLnaUlcn0gSW5jMRkwFwYDVQLExBx3
4 d3cuZGlnaWNLcn0uY29tMSAwHgYDVQ0EXdEaWdpO2VydCBHbG9iYWwgUm9mdCBD
5 QTAeFw0yMTA0MTQwMDAwMDBaFw0zMTA0MTMyMzU5NTlaME8xCzAIBGNVBAITVT
6 MRUwEwYDVQKExwEaWdpO2VydCBjbmMxKTANBgNVBAMTIERpZ2l0ZXI0IFRMuYBS
7 U0EGU0BMbjU2IDlwMjAgQ0EExMIIBIJANBgkqhkiG9w0BAQEFAAOCAQAMIIBCqKC
8 AQEAwUuzZUdwvN1PWNvsN03DuUfMNRNUpmRh8SuxKB+Uu3NySCiDt3+PE0J6a
9 qXodgojlEVbbHp9YwLHnLDQNLTKS4Vbl8XLfs7uHyIUDeSpSQWYQYE9XE0nw6Ddn
10 g9/n00tnTCJRtp80mRDtV1F0Ju39x8piLHMbfyOIJVNvwTRYAIue/+/+p1hJIInuW
11 raIKmxW8oHzf6VGolbdN+I2tIjLYrVJmuzHZ9bjPxXj1hJeRPg/cUJ9WI0dgLGB
12 Afr5yjK7tI4nyhfFK3TUgNaX3sNk+crOU6JWvHgXkkDA77SU+kFbn081lvZV21r
13 eacroiCGE7XOPUDTITAhk+qZ9QIDAQABo4IBGjCCAX4wEgYDVDR0TAQH/BAGwBgEB
14 /wTBADAAdBgNVHQ4EFgQt2u16qiqlIx56rTaD5iyxZV2ufQwHuYDVR0JB8gwFoAU
15 A9SQNVbRTLtmBKPIGxvdl7I90UVwdgYDVR0PAQH/BAQDAAGGMBOGA1udj0QMwBGQ
16 CCsgAQFBwMBBggrBgEFBQcDAjB2BggrBgEFBQcBAQRqMGggJAYIKWYBBQUHMAIG
17 GG0hdHA6Ly9vY3NwLnMzR2p1ZjZ0LmNvbTBABGgrBgEFBQcwAoYwAH0R+DovL2Nh
18 Y2YvdHMuzZGlnaWNLcn0uY29tL0RpZ2l0ZXI0R2xvYmFsU09vdENBLMRYdDBCgNV
19 HRRE0zASMDegNAzhjFodHRwOi8vY3JsMy5kaWdpY2VydCj5b20vRGlnaUlcnRH
20 Bg9iYXxz5b29080EuY3JsMD0GA1UdIAQ2MDQwCwY3YIZIAHY9bAIBMAcGBwwBAEB
21 MAQGBMeBDAAECATAIBgZng0wBAgiwCAYGZ4EMA0IDMA0GCsGSGTs30DEBCwUA4ATB
22 AQCAMs5eC9luWgOKr+HwhMwAjvqFc03axBmm9yt1QP6FCvrzMXi3CesAiVi6gl3z
23 ax3pf88Lu1clWdN0G/1s/dCYbbdxglvPbQtacdB73sRD2Cqk3pSBjL+-7j5nL3a7h
24 qG+fh/59t8BbIKuxT8bLZ1ldmzzp/2n3YwZ2fP9NsarA4h20ksudYbj/NhvfSbc
25 EXffPgK2fP0re3qGnm+499itcc+G33Mw+nur7SpzyKE0EXGLZy4Q4UafJbcm6e
26 ce1XR2FuAJKZTrEi9APCCcUZLM51Ke92sRKw25fh3oisus2Fk0H6ipjv3U/697E
27 A7skPPcw7+uvTPyLNhbZpvOk
28 -----END CERTIFICATE-----
29 --|
```

Figure 6.1.2. Issuer certificate

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Step 2

- Extract modulo from the issuer certificate with the command

```
openssl x509 -in c1.pem -noout -modulus
```

- Print all fields to find the e (public key) from the issuer certificate with the command

```
openssl x509 -in c1.pem -text -noout
```

Step 3

- Extract digital signature S from the server certificate

```
openssl x509 -in c0.pem -text -noout
```

- Saving the signature in the signature file (Figure 6.1.3)

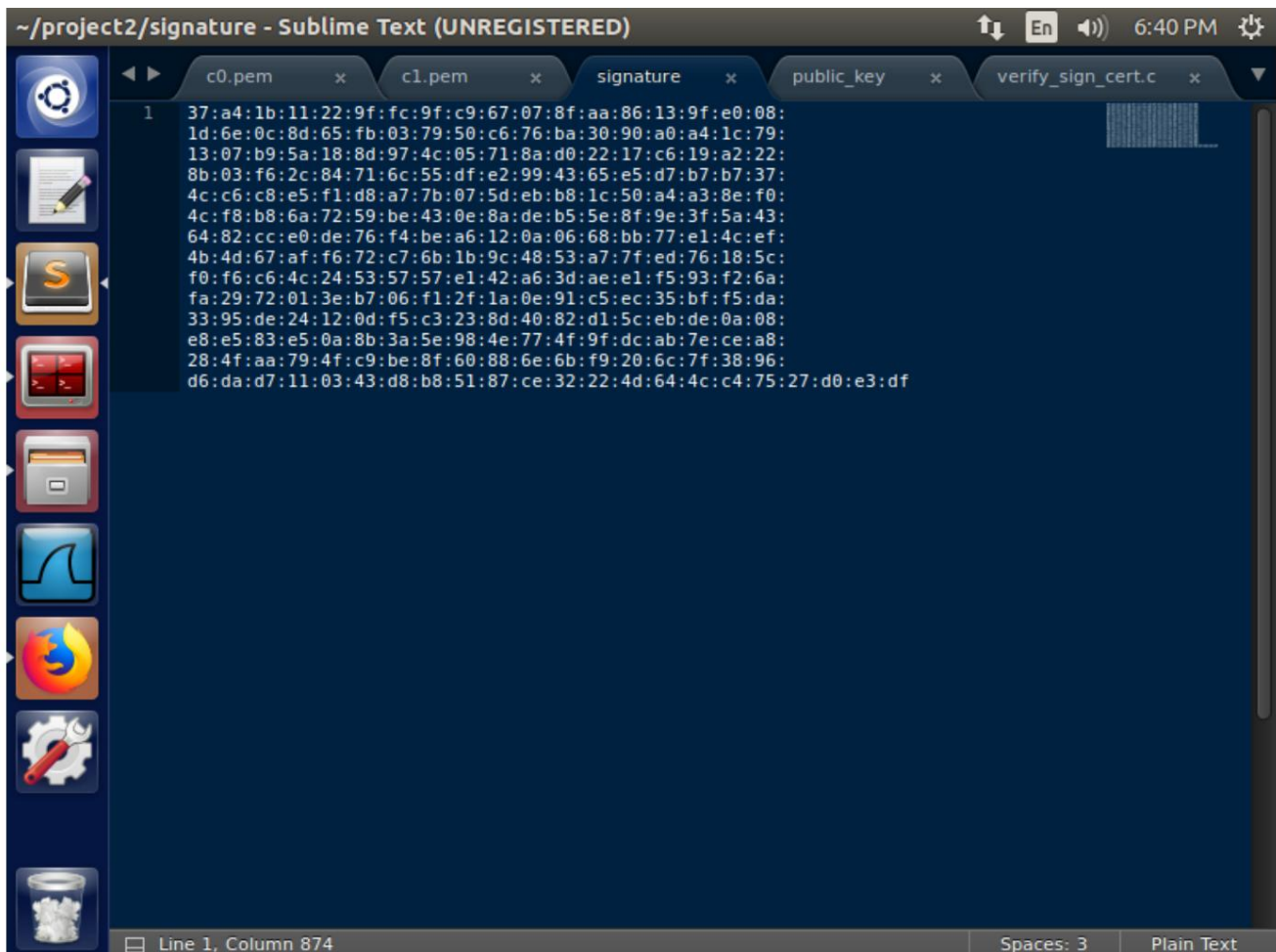


Figure 6.1.3. The digital signature from the server certificate

- Removing spaces and semicolons from data with the command

```
cat signature | tr -d '[:space:]'
```

Step 4

- Export the server certificate body with the command

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
openssl asn1parse -i -in c0.pem
```

- Save the server certificate body to a binary file c0_body.bin with the mandate

```
openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
```

- Calculation of the hash

```
sha256sum c0_body.bin
```

Step 5

After collecting the information, in this step we verified the digital signature of the certificate thanks to the `verify_sign_cert.c` program (Figure 6.1.4). In more detail:

`verify_sign_msg.c` (Figure 5.1.1) verifies the digital signature of the message by implementing mathematical operations with large numbers thanks to the BN function library that handles numbers that go beyond the limits of the computer architecture. In more detail:

in line 23 initializes the public key in the BN structure, e

```
BN_hex2bn(&e, "10001");
```

on line 27 initializes the modulo in the structure BN, N

```
BN_hex2bn(&N, "c14bb3654770bcdd4f58dbec9cedc366e51f311354ad4a66461f2c0aec6
407e52edcdcb90a20eddf3c4d09e9aa97a1d8288e51156db1e9f58c251e72c340d2ed292
e156cbf1795fb3bb87ca25037b9a52416610604f571349f0e8376783dfe7d34b674c2251a
6df0e9910ed57517426e27dc7ca622e131b7f238825536fc13458008b84fff8bea7584922
7b96ada2889b15bca07cdf951a8d5b0ed37e236b4824b62b5499aecc767d6e33ef5e3d61
25e44f1bf71427d58840380b18101faf9ca32bbb48e278727c52b74d4a8d697dec364f9ca
ce53a256bc78178e490329aefb494fa415b9cef25c19576d6b79a72ba2272013b5d03d40d 321300793ea99f5");
```

on line 31 initializes the digital signature of the certificate in the structure BN, S

```
BN_hex2bn(&S, "37a41b11229ffc9fc967078faa86139fe0081d6e0c8d65fb037950c676b
a3090a0a41c791307b95a188d974c05718ad02217c619a2228b03f62c84716c55dfe29943
65e5d7b7b7374cc6c8e5f1d8a77b075debb81c50a4a38ef04cf8b86a7259be430e8adeb55
e8f9e3f5a436482cce0de76f4bea6120a0668bb77e14cef4b4d67aff672c76b1b9c4853a7
7fed76185cf0f6c64c24535757e142a63dae1f593f26afa2972013eb706f12f1a0e91c5e
c35bff5da3395de24120df5c3238d4082d15cebde0a08e8e583e50a8b3a5e984e774f9fdc
ab7ecea8284faa794fc9be8f60886e6bf9206c7f3896d6dad7110343d8b85187ce32224d6
44cc47527d0e3df");
```

in line 35 initializes the hash value, encoded in hexadecimal format in the BN structure, H

```
BN_hex2bn(&H, "ed799631340493b633ba23ec5748dcae2ae1eef862fab54b369c044e2aa d4a63");
```

in line 40 it calculates the mathematical operation S , that $e \bmod N$ and assigns the result to the structure BN, D, is, it calculates the decryption of the digital signature of the certificate

```
BN_mod_exp(D, S, e, N, ctx);
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```

verify_sign_cert.c x
1  #include <stdio.h>
2  #include <openssl/bn.h>
3
4  #define NBITS 256
5
6  void printBN(char *msg, BIGNUM *a);
7
8  int main(int argc, char *argv[])
9  {
10     int areEqual;
11
12     /*
13      * Δημιουργία των δομών τύπου BN
14      */
15     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
16     BIGNUM *D = BN_new(); // Μήνυμα
17     BIGNUM *H = BN_new(); // Επαλήθευση υπογραφής
18     BIGNUM *S = BN_new(); // Υπογραφή μηνύματος
19     BIGNUM *e = BN_new(); // Δημόσιο κλειδί (e, N)
20     BIGNUM *N = BN_new(); // Modulo N
21
22     /*
23      * Αρχικοποίηση του δημόσιου κλειδιού (e, N) από το πιστοποιητικό του εκδότη
24      */
25     BN_hex2bn(&e, "10001");
26
27     /*
28      * Αρχικοποίηση του modulo N από το πιστοποιητικό του εκδότη
29      */
30     BN_hex2bn(&N, "c14bb3654770bcdd4f58dbec9cedc366e51f311354ad4a66461f2c0aec6407e5");
31
32     /*
33      * Αρχικοποίηση της υπογραφής από το πιστοποιητικό του server
34      */
35     BN_hex2bn(&S, "37a41b11229ffc9fc967078faa86139fe0081d6e0c8d65fb037950c676ba3096");
36
37     /*
38      * Αρχικοποίηση του ψηφιακού αποτυπώματος H μέσω υπολογισμού από την συνάρτηση hash
39      */
40     BN_hex2bn(&H, "ed799631340493b633ba23ec5748dcae2ae1eef862fab54b369c044e2aad4a63");
41
42     /*
43      * Υπολογισμός της "αποκρυπτογράφησης" της ψηφιακής υπογραφής από
44      * το πιστοποιητικό του server με τον τύπο  $D = S^e \bmod N$ 
45      */
46     BN_mod_exp(D, S, e, N, ctx);
47
48     /*
49      * Εκτύπωση του ψηφιακού αποτυπώματος
50      */
51     printBN("Hash \t\t =", H);
52
53     /*
54      * Εκτύπωση της "αποκρυπτογραφημένης" ψηφιακής υπογραφής
55      */
56     printBN("Digest \t\t =", D);
57
58     /*
59      * Αποδέσμευση των δομών τύπου BN από την μνήμη
60      */
61     BN_CTX_free(ctx);
62     BN_free(D);
63     BN_free(H);
64     BN_free(S);
65     BN_free(e);
66     BN_free(N);
67
68     return 0;
69 }
70
71 /*
72 * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
73 */
74 void printBN(char *msg, BIGNUM *a)
75 {
76     char *number_str = BN_bn2hex(a);
77     printf("%s %s\n\n", msg, number_str);
78     OPENSSL_free(number_str);
79 }

```

Figure 6.1.5. The verify_sing_cert.c program

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Then, we compiled the code with the parameter

```
-lcrypto
```

where it links to the OpenSSL BN library of large number manipulation functions.

The program compilation command is:

```
gcc verify_sign_cert.c -o verify_sign_cert -lcrypto
```

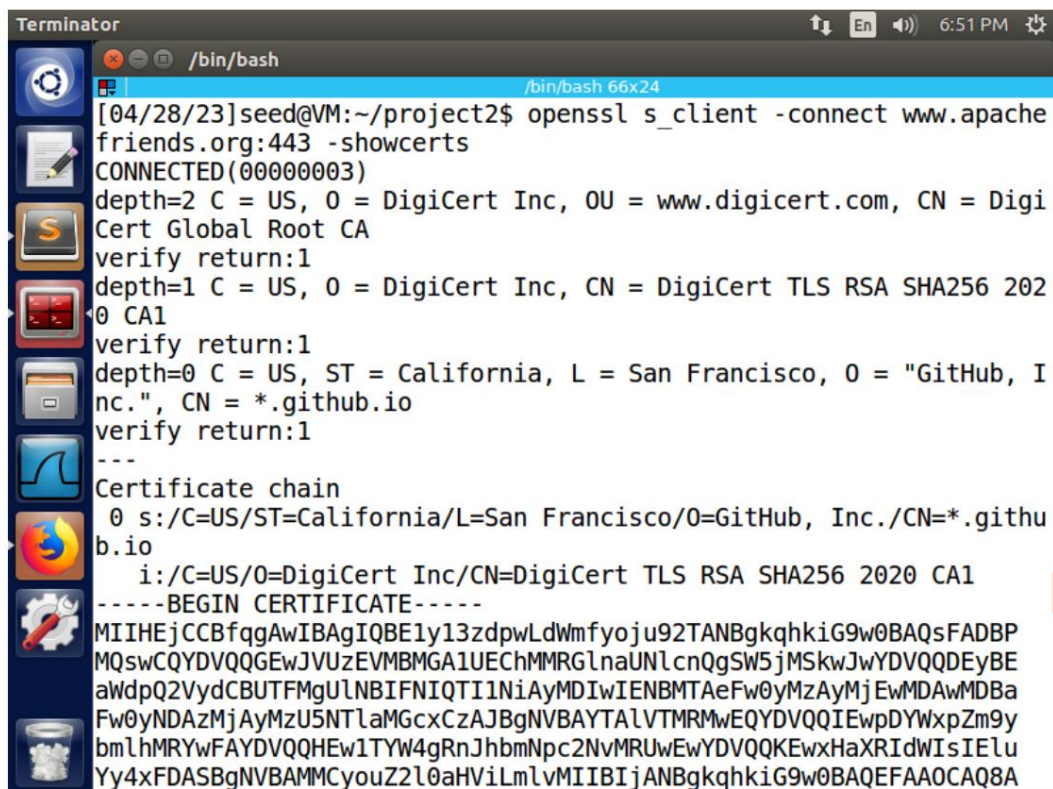
and since there were no syntax errors, the command output was empty and the executable file `verify_sign_cert` was created.

Then, we ran the `verify_sign_msg` program with the command:

```
./verify_sign_cert
```

6.2 Results

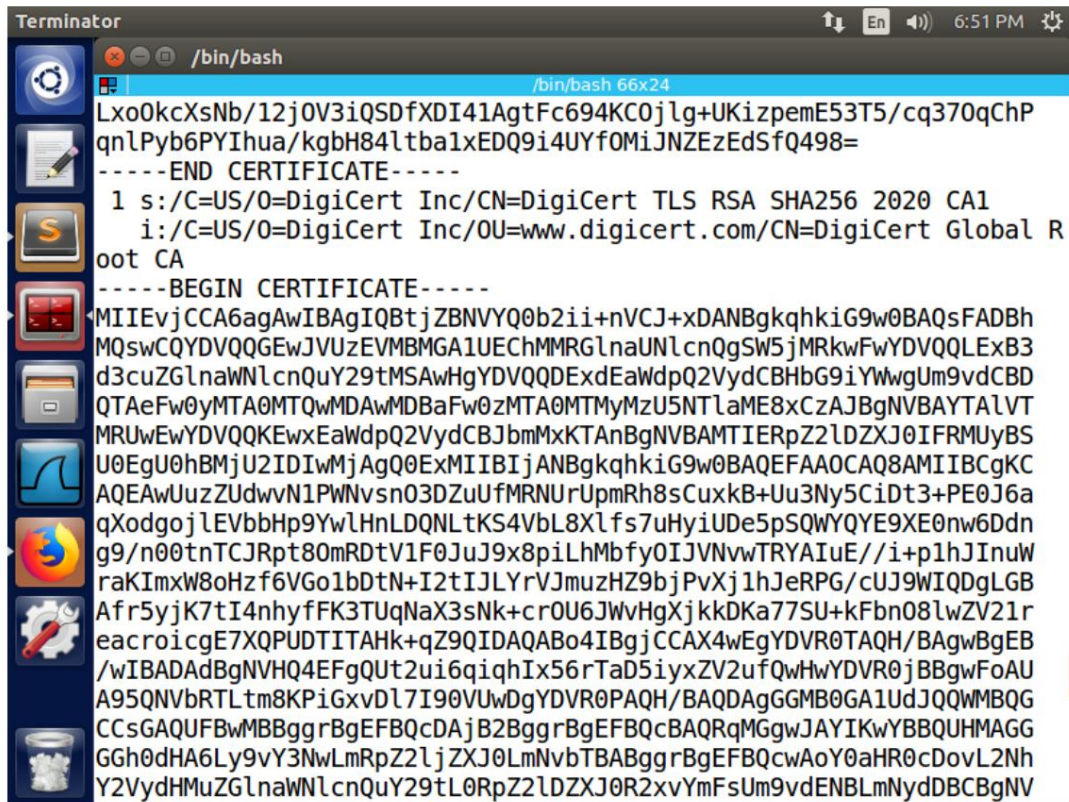
Step 1



```
Terminator /bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl s_client -connect www.apache
friends.org:443 -showcerts
CONNECTED(00000003)
depth=2 C = US, O = DigiCert Inc, OU = www.digicert.com, CN = Digi
Cert Global Root CA
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert TLS RSA SHA256 202
0 CA1
verify return:1
depth=0 C = US, ST = California, L = San Francisco, O = "GitHub, I
nc.", CN = *.github.io
verify return:1
---
Certificate chain
 0 s:/C=US/ST=California/L=San Francisco/O=GitHub, Inc./CN=*.githu
b.io
 1 i:/C=US/O=DigiCert Inc/CN=DigiCert TLS RSA SHA256 2020 CA1
-----BEGIN CERTIFICATE-----
MIIHEjCCBfqgAwIBAgIQBE1y13zdpwLdWmfyoju92TANBgkqhkiG9w0BAQsFADBP
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMzMmRGlbnUNlcnQgSw5jMSkwJwYDVQQDEyBE
aWdpQ2VydCBUTFMgUlnBIFNIQTI1NiAyMDIwIENBMTAeFw0yMzAyMjEwMDAwMDBa
Fw0yNDAzMjAyMzU5NTlaMGcxOzAJBgNVBAYTAlVTMRMwEQYDVQQIEwpDYWxpZm9y
bmVhbmR5bWVudDQHEw1TYW4gRnJhbmNpc2NvMRUwEwYDVQQKEwxaXRIIdWIsIElu
Yy4xFDASBgNVBAMMCyouZ2l0aHViLmlvMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A
```

Figure 6.2.1. Downloading the server certificate

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



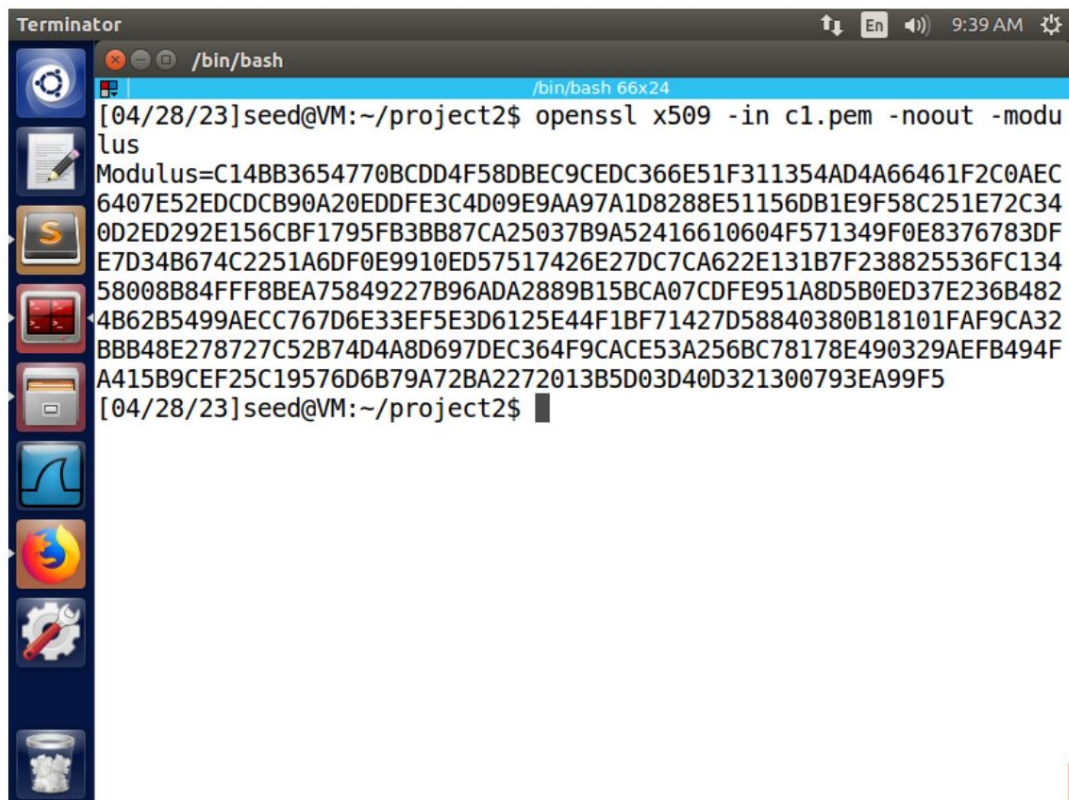
```

Terminator
/bin/bash
/bin/bash 66x24
Lxo0kcXsNb/12j0V3iQSDfXDI41AgtFc694KC0jlg+UKizpemE53T5/cq370qChP
qnLPyb6PYIhua/kgbH84ltba1xEDQ9i4UYf0MiJNZEzEdSfQ498=
-----END CERTIFICATE-----
1 s:/C=US/O=DigiCert Inc/CN=DigiCert TLS RSA SHA256 2020 CA1
i:/C=US/O=DigiCert Inc/OU=www.digicert.com/CN=DigiCert Global R
oot CA
-----BEGIN CERTIFICATE-----
MIIEVjCCA6agAwIBAgIQBtjZBNVYQ0b2ii+nVCJ+xDANBgkqhkiG9w0BAQsFADBh
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNLcnQgSW5jMRkwFwYDVQLExB3
d3cuZGlnaWNlcnQuY29tMSAwHgYDVQQDEXdEaWdpQ2VydCBHbG9iYWwgUm9vdCBD
QTAEFw0yMTA0MTQwMDAwMDBaFw0zMTA0MTMyMzU5NTlaME8xCzAJBgNVBAYTAlVT
MRUwEwYDVQQKEwxEaWdpQ2VydCBJbmMxKTAnBgNVBAMTIERpZ2l0ZXJ0IFR1b3R1
U0EgU0hBMjU2IDIwMjU5MTA0MTQwMDAwMDBaFw0zMTA0MTMyMzU5NTlaME8xCzAJ
AQEAWUuzZUdwN1PWNVsnO3DZuUfMRNURUpmRh8sCuxkB+Uu3Ny5CiDt3+PE0J6a
qXodgojLEVbbHp9YwLHnLDQNLtKS4Vbl8Xlfs7uHyiUDe5pSQWYQYE9XE0nw6Ddn
g9/n00tnTCJRpt80mRDtV1F0JuJ9x8piLhMbfy0IJVNvwTRYAIuE//i+p1hJInuW
raKImxW8oHzf6VGo1bDn+I2tIJLYrVJmuzHZ9bjPvXj1hJeRPG/cUJ9WIQDgLGb
Afr5yjk7tI4nhyfFK3TUqNaX3sNk+crOU6JWvHgXjkkDKa77SU+kFbn08lwZV21r
eacroicgE7XQPUDTITAHk+qZ9QIDAQABo4IBgjCCAX4wEgYDVR0TAQH/BAgwBgEB
/wIBADAdBgNVHQ4EFgQt2ui6qihIx56rTaD5iyxZV2ufQwHwYDVR0jBBgwFoAU
A95QNvbRtLtm8KPiGxvDl7I90VUwDgYDVR0PAQH/BAQDAgGMB0GA1UdJQQWMBQG
CCsGAQUFBwMBBggrBgEFBQcDAjB2BggrBgEFBQcBAQRqMGgwJAYIKwYBBQUHMAAG
GGh0dHA6Ly9vY3NwLmRpZ2l0ZXJ0LmNvbTBABggrBgEFBQcwAoY0aHR0cDovL2Nh
Y2VydHMuZGlnaWNlcnQuY29tL0R0Z2l0ZXJ0R2xvYmFsUm9vdENBLmNydDBCBgNV

```

Figure 6.2.2. Obtaining the issuer's certificate

Step 2



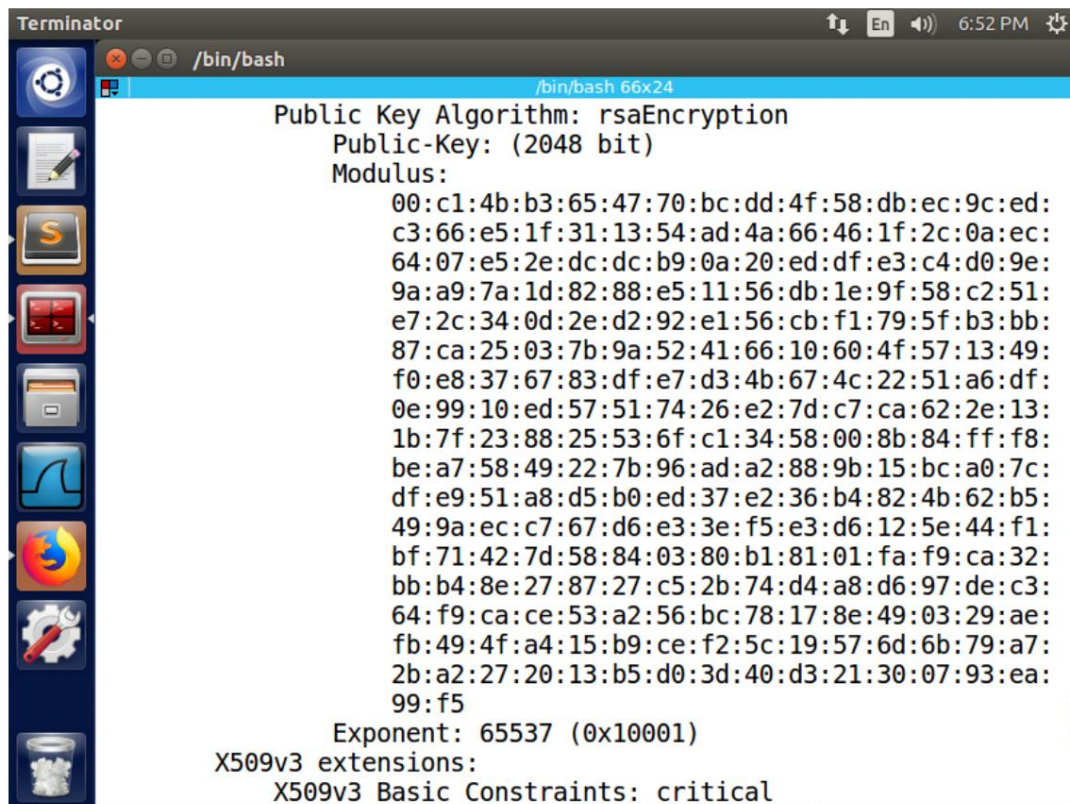
```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl x509 -in c1.pem -noout -modu
lus
Modulus=C14BB3654770BCDD4F58DBEC9CEDC366E51F311354AD4A66461F2C0AEC
6407E52EDCDB90A20EDDFE3C4D09E9AA97A1D8288E51156DB1E9F58C251E72C34
0D2ED292E156CBF1795FB3BB87CA25037B9A52416610604F571349F0E8376783DF
E7D34B674C2251A6DF0E9910ED57517426E27DC7CA622E131B7F238825536FC134
58008B84FFF8BEA75849227B96ADA2889B15BCA07CDFE951A8D5B0ED37E236B482
4B62B5499AEC767D6E33EF5E3D6125E44F1BF71427D58840380B18101FAF9CA32
BBB48E278727C52B74D4A8D697DEC364F9CACE53A256BC78178E490329AEFB494F
A415B9CEF25C19576D6B79A72BA2272013B5D03D40D321300793EA99F5
[04/28/23]seed@VM:~/project2$

```

Figure 6.2.3. Extracting the modulo N from the issuer certificate

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



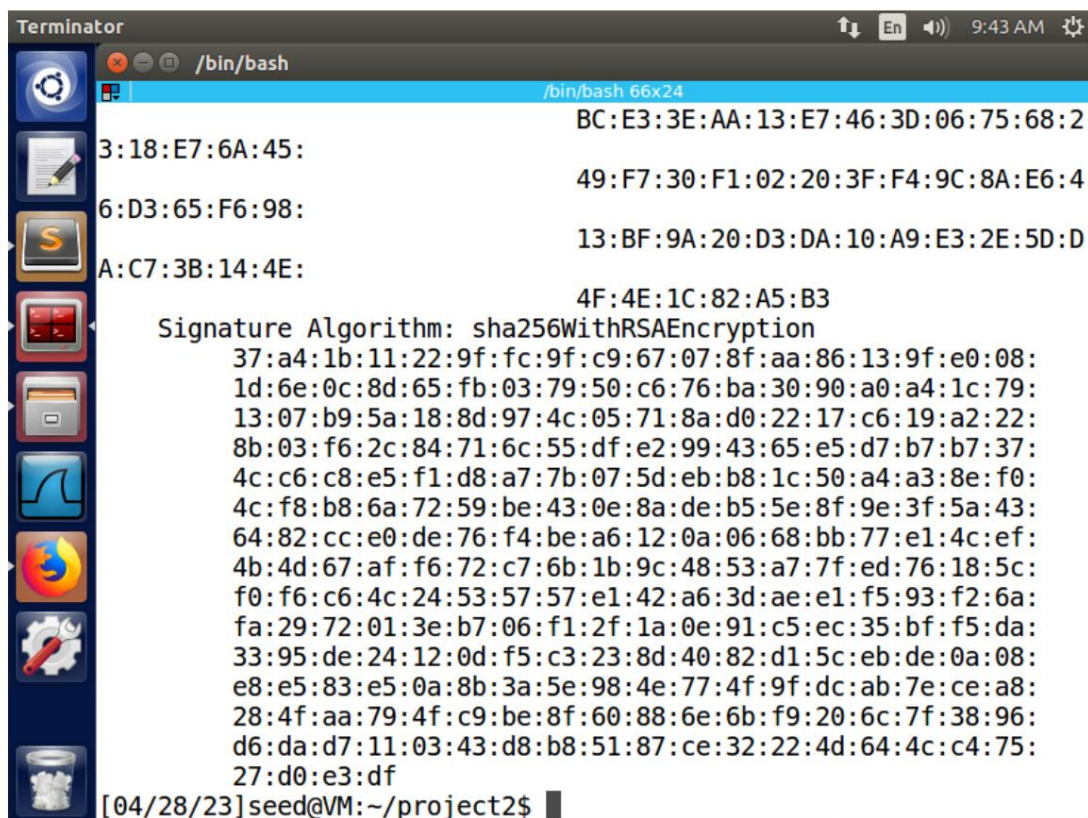
```

Terminator
/bin/bash
/bin/bash 66x24
Public Key Algorithm: rsaEncryption
Public-Key: (2048 bit)
Modulus:
  00:c1:4b:b3:65:47:70:bc:dd:4f:58:db:ec:9c:ed:
  c3:66:e5:1f:31:13:54:ad:4a:66:46:1f:2c:0a:ec:
  64:07:e5:2e:dc:dc:b9:0a:20:ed:df:e3:c4:d0:9e:
  9a:a9:7a:1d:82:88:e5:11:56:db:1e:9f:58:c2:51:
  e7:2c:34:0d:2e:d2:92:e1:56:cb:f1:79:5f:b3:bb:
  87:ca:25:03:7b:9a:52:41:66:10:60:4f:57:13:49:
  f0:e8:37:67:83:df:e7:d3:4b:67:4c:22:51:a6:df:
  0e:99:10:ed:57:51:74:26:e2:7d:c7:ca:62:2e:13:
  1b:7f:23:88:25:53:6f:c1:34:58:00:8b:84:ff:f8:
  be:a7:58:49:22:7b:96:ad:a2:88:9b:15:bc:a0:7c:
  df:e9:51:a8:d5:b0:ed:37:e2:36:b4:82:4b:62:b5:
  49:9a:ec:c7:67:d6:e3:3e:f5:e3:d6:12:5e:44:f1:
  bf:71:42:7d:58:84:03:80:b1:81:01:fa:f9:ca:32:
  bb:b4:8e:27:87:27:c5:2b:74:d4:a8:d6:97:de:c3:
  64:f9:ca:ce:53:a2:56:bc:78:17:8e:49:03:29:ae:
  fb:49:4f:a4:15:b9:ce:f2:5c:19:57:6d:6b:79:a7:
  2b:a2:27:20:13:b5:d0:3d:40:d3:21:30:07:93:ea:
  99:f5
Exponent: 65537 (0x10001)
X509v3 extensions:
X509v3 Basic Constraints: critical

```

Figure 6.2.4. Extracting the public key e from the issuer certificate

Step 3



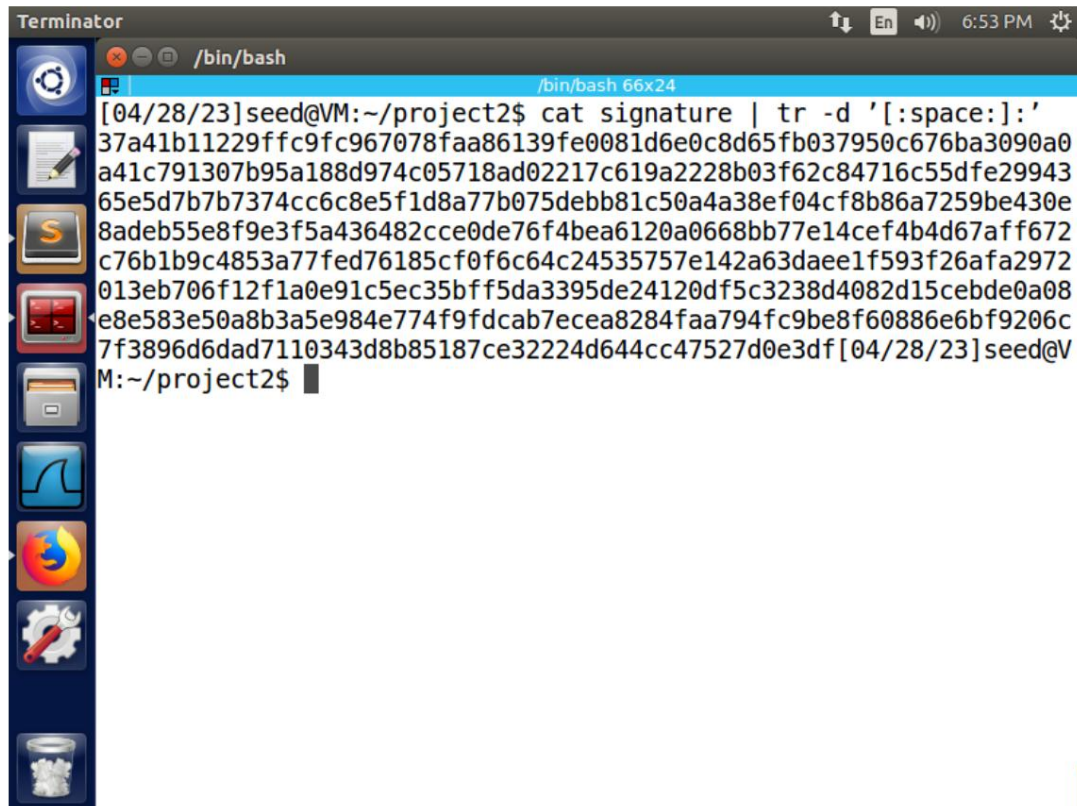
```

Terminator
/bin/bash
/bin/bash 66x24
BC:E3:3E:AA:13:E7:46:3D:06:75:68:2
3:18:E7:6A:45:
49:F7:30:F1:02:20:3F:F4:9C:8A:E6:4
6:D3:65:F6:98:
13:BF:9A:20:D3:DA:10:A9:E3:2E:5D:D
A:C7:3B:14:4E:
4F:4E:1C:82:A5:B3
Signature Algorithm: sha256WithRSAEncryption
  37:a4:1b:11:22:9f:fc:9f:c9:67:07:8f:aa:86:13:9f:e0:08:
  1d:6e:0c:8d:65:fb:03:79:50:c6:76:ba:30:90:a0:a4:1c:79:
  13:07:b9:5a:18:8d:97:4c:05:71:8a:d0:22:17:c6:19:a2:22:
  8b:03:f6:2c:84:71:6c:55:df:e2:99:43:65:e5:d7:b7:b7:37:
  4c:c6:c8:e5:f1:d8:a7:7b:07:5d:eb:b8:1c:50:a4:a3:8e:f0:
  4c:f8:b8:6a:72:59:be:43:0e:8a:de:b5:5e:8f:9e:3f:5a:43:
  64:82:cc:e0:de:76:f4:be:a6:12:0a:06:68:bb:77:e1:4c:ef:
  4b:4d:67:af:f6:72:c7:6b:1b:9c:48:53:a7:7f:ed:76:18:5c:
  f0:f6:c6:4c:24:53:57:57:e1:42:a6:3d:ae:e1:f5:93:f2:6a:
  fa:29:72:01:3e:b7:06:f1:2f:1a:0e:91:c5:ec:35:bf:f5:da:
  33:95:de:24:12:0d:f5:c3:23:8d:40:82:d1:5c:eb:de:0a:08:
  e8:e5:83:e5:0a:8b:3a:5e:98:4e:77:4f:9f:dc:ab:7e:ce:a8:
  28:4f:aa:79:4f:c9:be:8f:60:88:6e:6b:f9:20:6c:7f:38:96:
  d6:da:d7:11:03:43:d8:b8:51:87:ce:32:22:4d:64:4c:c4:75:
  27:d0:e3:df
[04/28/23]seed@VM:~/project2$

```

Figure 6.2.5. Extracting the digital signature from the server certificate

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



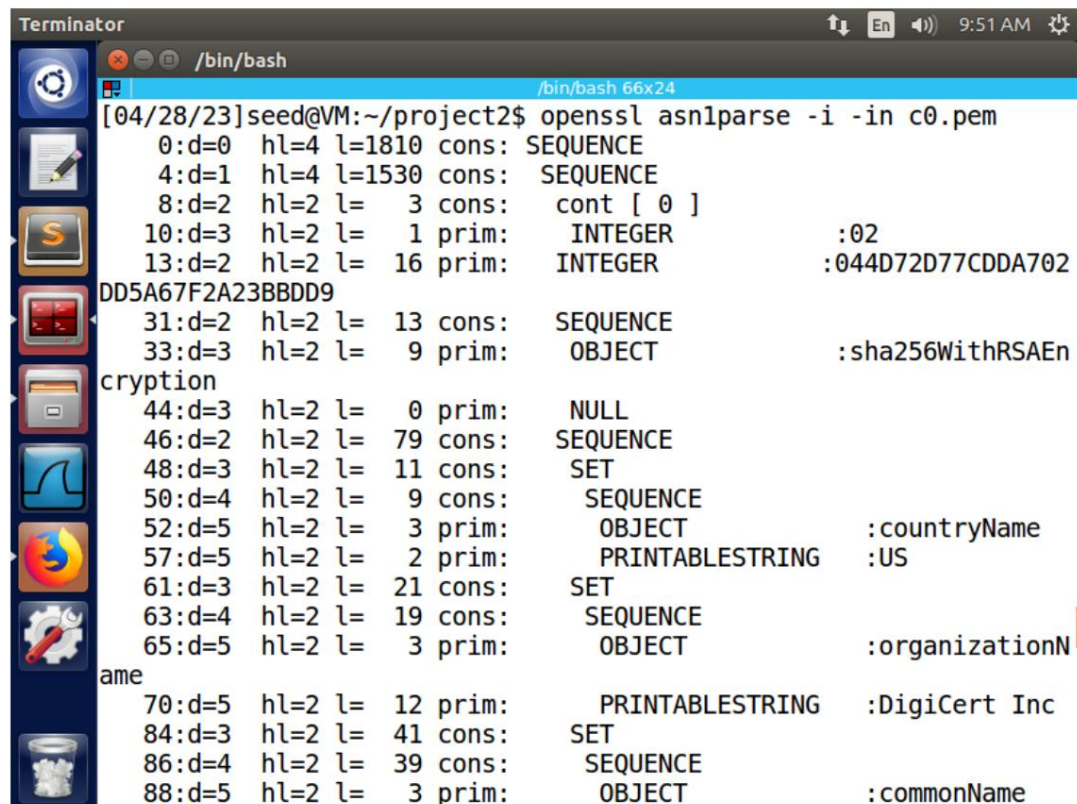
```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ cat signature | tr -d '[:space:]:'
37a41b11229ffc9fc967078faa86139fe0081d6e0c8d65fb037950c676ba3090a0
a41c791307b95a188d974c05718ad02217c619a2228b03f62c84716c55dfe29943
65e5d7b7b7374cc6c8e5f1d8a77b075debb81c50a4a38ef04cf8b86a7259be430e
8adeb55e8f9e3f5a436482cce0de76f4bea6120a0668bb77e14cef4b4d67aff672
c76b1b9c4853a77fed76185cf0f6c64c24535757e142a63dae1f593f26afa2972
013eb706f12f1a0e91c5ec35bff5da3395de24120df5c3238d4082d15cebde0a08
e8e583e50a8b3a5e984e774f9fdcab7ecea8284faa794fc9be8f60886e6bf9206c
7f3896d6dad7110343d8b85187ce32224d644cc47527d0e3df[04/28/23]seed@V
M:~/project2$

```

Figure 6.2.6. The digital signature of the certificate in a single sequence

Step 4

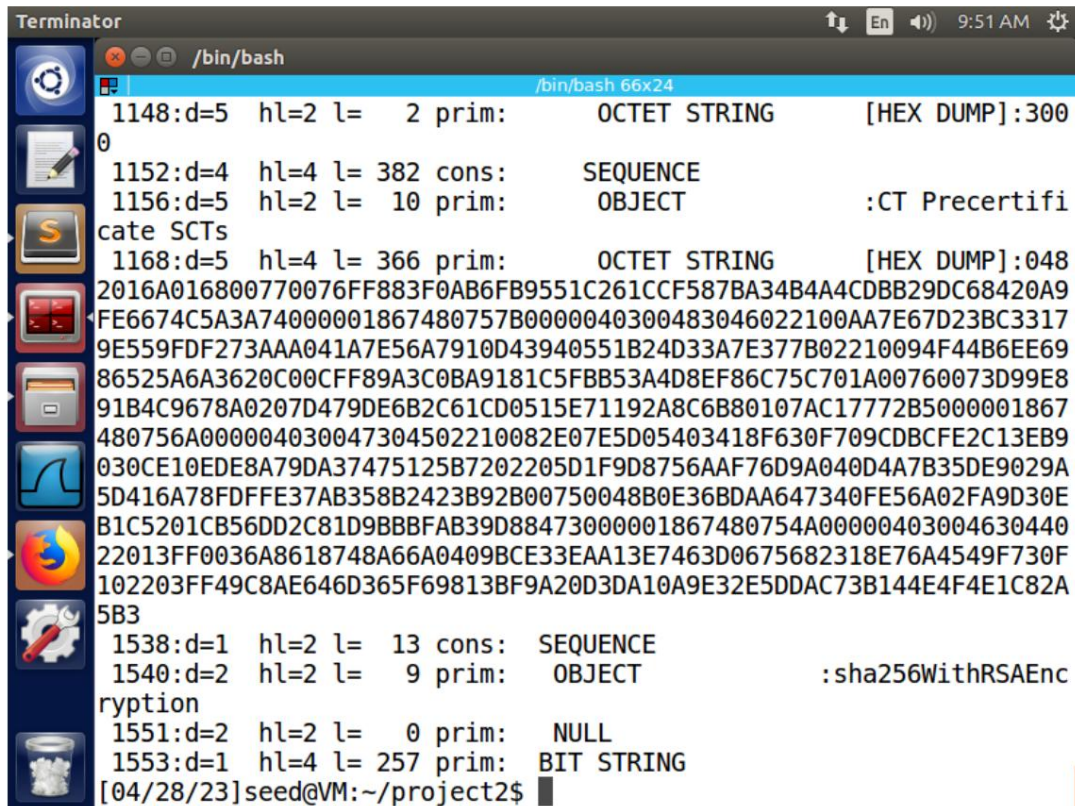


```

Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl asn1parse -i -in c0.pem
0:d=0 hl=4 l=1810 cons: SEQUENCE
4:d=1 hl=4 l=1530 cons: SEQUENCE
8:d=2 hl=2 l= 3 cons: cont [ 0 ]
10:d=3 hl=2 l= 1 prim: INTEGER :02
13:d=2 hl=2 l= 16 prim: INTEGER :044D72D77CDDA702
DD5A67F2A23BBDD9
31:d=2 hl=2 l= 13 cons: SEQUENCE
33:d=3 hl=2 l= 9 prim: OBJECT :sha256WithRSAEn
cryption
44:d=3 hl=2 l= 0 prim: NULL
46:d=2 hl=2 l= 79 cons: SEQUENCE
48:d=3 hl=2 l= 11 cons: SET
50:d=4 hl=2 l= 9 cons: SEQUENCE
52:d=5 hl=2 l= 3 prim: OBJECT :countryName
57:d=5 hl=2 l= 2 prim: PRINTABLESTRING :US
61:d=3 hl=2 l= 21 cons: SET
63:d=4 hl=2 l= 19 cons: SEQUENCE
65:d=5 hl=2 l= 3 prim: OBJECT :organizationN
ame
70:d=5 hl=2 l= 12 prim: PRINTABLESTRING :DigiCert Inc
84:d=3 hl=2 l= 41 cons: SET
86:d=4 hl=2 l= 39 cons: SEQUENCE
88:d=5 hl=2 l= 3 prim: OBJECT :commonName

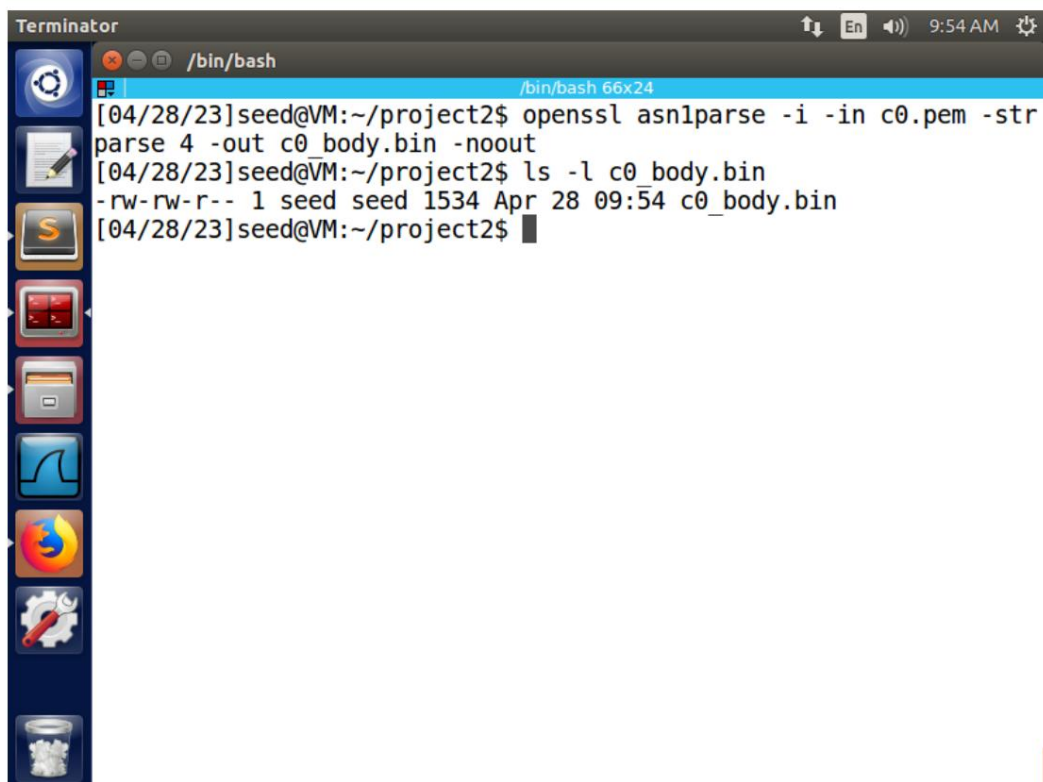
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator /bin/bash
/bin/bash 66x24
1148:d=5 hl=2 l= 2 prim: OCTET STRING [HEX DUMP]:300
0
1152:d=4 hl=4 l= 382 cons: SEQUENCE
1156:d=5 hl=2 l= 10 prim: OBJECT :CT Precertifi
cate SCTs
1168:d=5 hl=4 l= 366 prim: OCTET STRING [HEX DUMP]:048
2016A016800770076FF883F0AB6FB9551C261CCF587BA34B4A4CDBB29DC68420A9
FE6674C5A3A74000001867480757B0000040300483046022100AA7E67D23BC3317
9E559FDF273AAA041A7E56A7910D43940551B24D33A7E377B02210094F44B6EE69
86525A6A3620C00CFF89A3C0BA9181C5FBB53A4D8EF86C75C701A00760073D99E8
91B4C9678A0207D479DE6B2C61CD0515E71192A8C6B80107AC17772B5000001867
480756A000004030047304502210082E07E5D05403418F630F709CDBCFC2C13EB9
030CE10EDE8A79DA37475125B7202205D1F9D8756AAF76D9A040D4A7B35DE9029A
5D416A78FDFFE37AB358B2423B92B00750048B0E36BDAA647340FE56A02FA9D30E
B1C5201CB56DD2C81D9BBBFAB39D88473000001867480754A00000403004630440
22013FF0036A8618748A66A0409BCE33EAA13E7463D0675682318E76A4549F730F
102203FF49C8AE646D365F69813BF9A20D3DA10A9E32E5DDAC73B144E4F4E1C82A
5B3
1538:d=1 hl=2 l= 13 cons: SEQUENCE
1540:d=2 hl=2 l= 9 prim: OBJECT :sha256WithRSAEnc
ryption
1551:d=2 hl=2 l= 0 prim: NULL
1553:d=1 hl=4 l= 257 prim: BIT STRING
[04/28/23]seed@VM:~/project2$
```

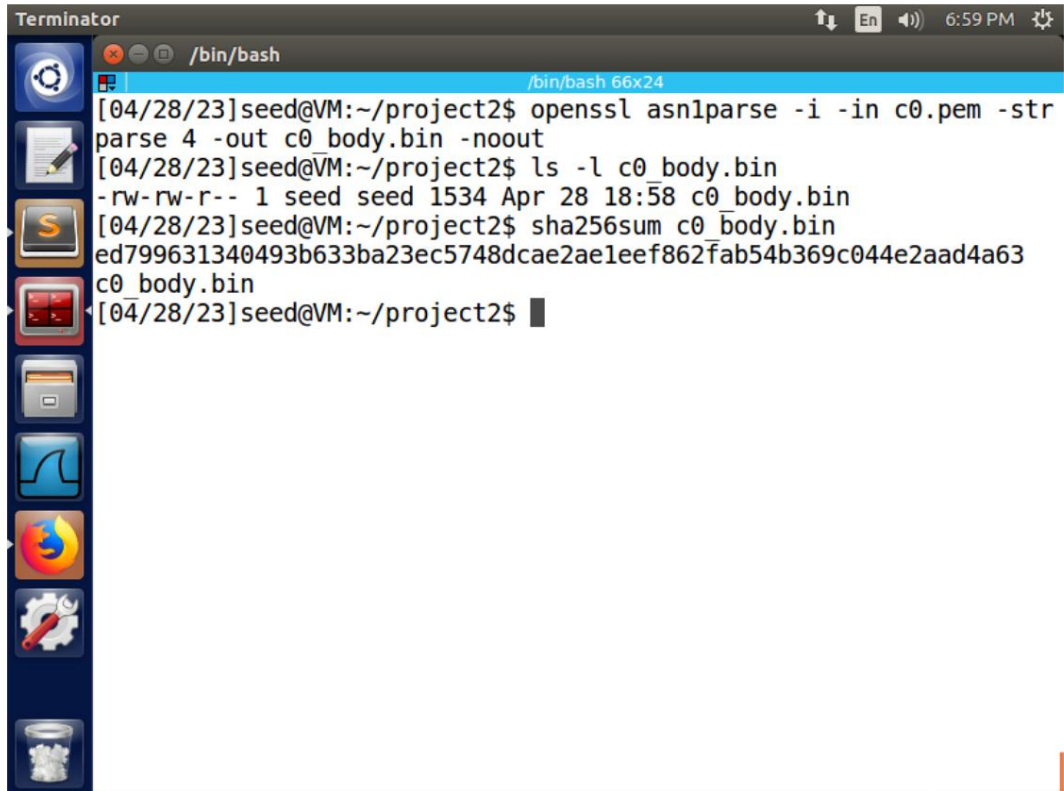
Figure 6.2.7. The certificate body



```
Terminator /bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl asn1parse -i -in c0.pem -str
parse 4 -out c0_body.bin -noout
[04/28/23]seed@VM:~/project2$ ls -l c0_body.bin
-rw-rw-r-- 1 seed seed 1534 Apr 28 09:54 c0_body.bin
[04/28/23]seed@VM:~/project2$
```

Figure 6.2.8. Exporting the certificate body

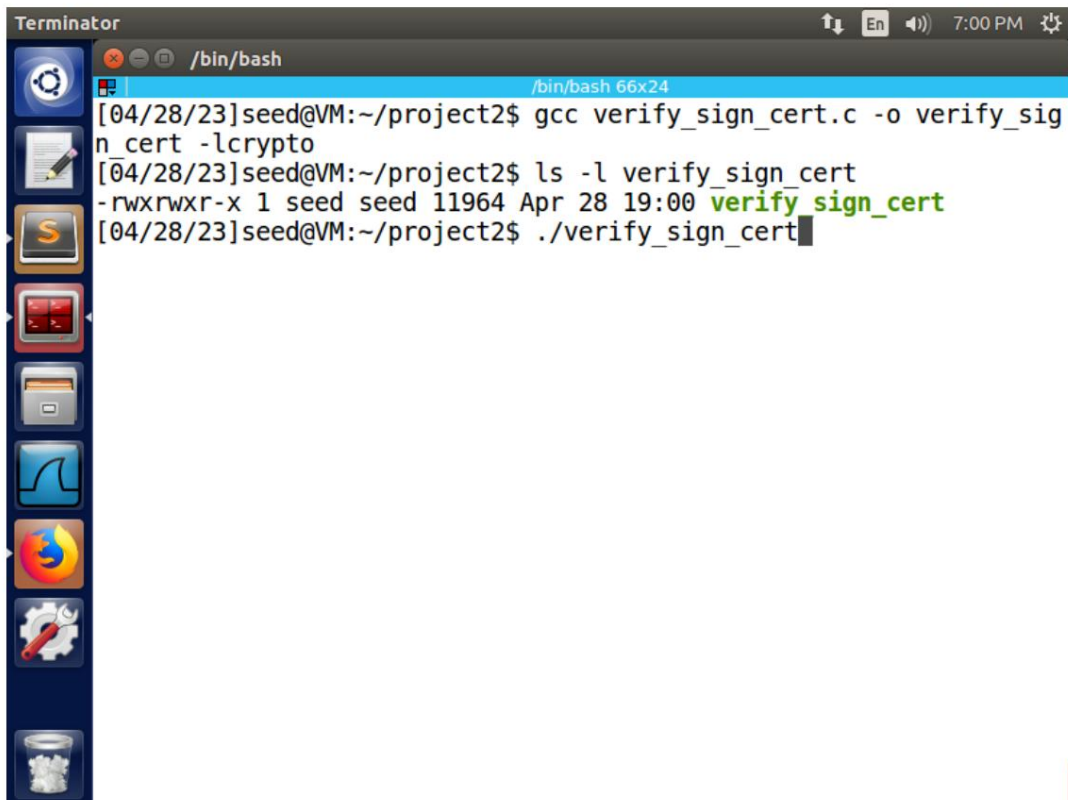
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator                                     6:59 PM
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl asn1parse -i -in c0.pem -str
parse 4 -out c0_body.bin -noout
[04/28/23]seed@VM:~/project2$ ls -l c0_body.bin
-rw-rw-r-- 1 seed seed 1534 Apr 28 18:58 c0_body.bin
[04/28/23]seed@VM:~/project2$ sha256sum c0_body.bin
ed799631340493b633ba23ec5748dcae2ae1eef862fab54b369c044e2aad4a63
c0_body.bin
[04/28/23]seed@VM:~/project2$
```

Figure 6.2.9. Calculating the hash

Step 5



```
Terminator                                     7:00 PM
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ gcc verify_sign_cert.c -o verify_sig
n_cert -lcrypto
[04/28/23]seed@VM:~/project2$ ls -l verify_sign_cert
-rwxrwxr-x 1 seed seed 11964 Apr 28 19:00 verify_sign_cert
[04/28/23]seed@VM:~/project2$ ./verify_sign_cert
```

Figure 6.2.10. Compiling and running the program verify_sign_cert.c

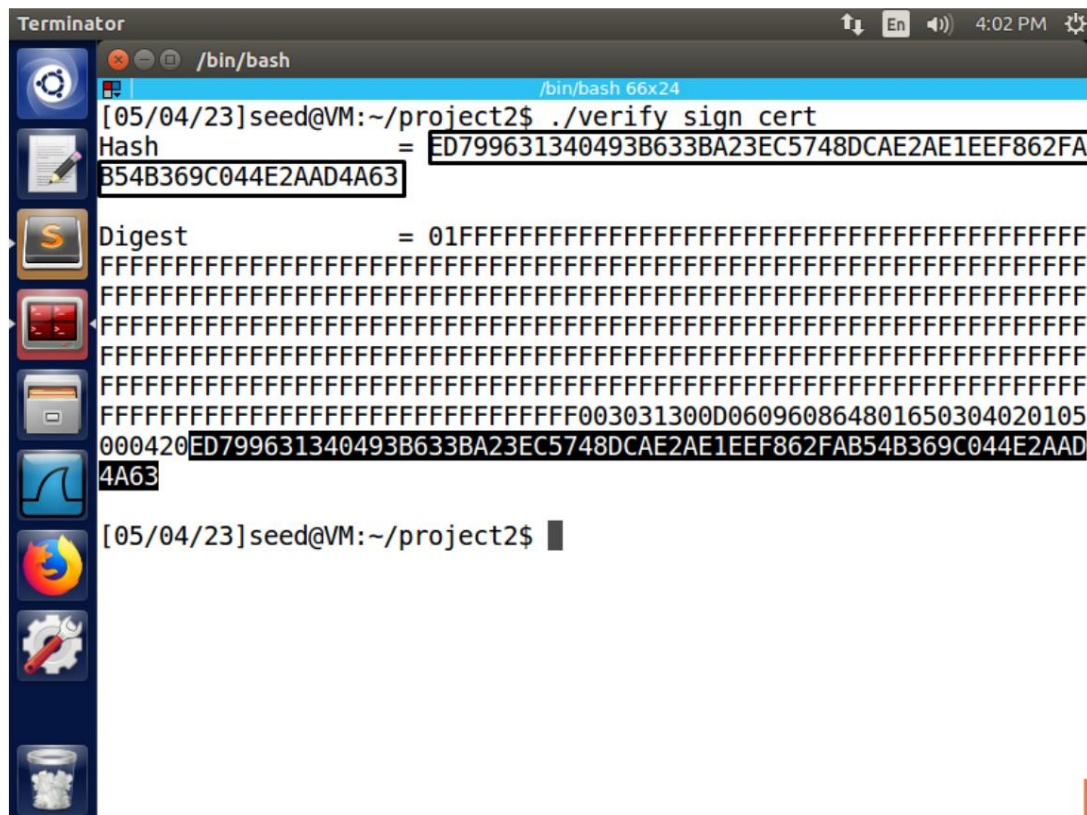


Figure 6.2.11. Running the program `verify_sign_cert.c`

6.3 Observations

The decryption of the digital signature was calculated by the mathematical formula $D = S^e \bmod N$. By executing the program `verify_sing_cert.c` and with the information we collected in the aforementioned steps, we observed that the signature is valid, as the decrypted digital signature contains the hash value we calculated in Step 4 at the end (Figure 6.2.11).

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



Thank you for your attention. Ευχαριστώ για την προσοχή σας.

